



**T. Y. B. TECH
INFORMATION TECHNOLOGY
(2026 COURSE)**

**LABORATORY MANUAL
FOR**

**Artificial Intelligence and
Machine Learning Laboratory
(AIMLL)**

SEMESTER – III

[Subject code: 2635PC32]

[Prepared By]

Mrs. Sumitra A. Jakhete

Dr. Anant M. Bagade

Mr. Vinit R. Tribhuvan

VISION

SCTR's Pune Institute of Computer Technology envisions to be a centre of excellence in higher technical education and research, producing global leaders in the field of technology.

MISSION

- Provide rigorous academic training that equips students to excel in their professional careers with the development of universal human values.
- Foster research and innovation culture amongst faculty and students by encouraging interdisciplinary collaborations, thereby contributing to technological advancements.
- Nurture leadership qualities in students through development of analytical thinking, problem-solving, and effective communication skills.

DEPARTMENT VISION AND MISSION

VISION

The department aspires to be a transformative force in Information Technology education and research, developing globally competitive professionals.

MISSION

- Inculcate an environment of academic rigor in Information Technology education promoting critical thinking, creativity, and problem-solving skills.
- Foster research and industry collaboration by promoting multidisciplinary engagement among students and faculty, driving technological advancements to address societal challenges.
- Cultivate skilled professionals who embody effective communication, leadership, and ethical values.


Third Year B. Tech (T. Y B. Tech) AY (2026-27)
Information Technology (IT)
[2635PC32]: Artificial Intelligence and Machine Learning Laboratory (AIMLL)

Semester	Credits	Teaching Scheme	Examination Scheme
5	01	P: 02 Hrs. / Week	CIE (TW): 25 Marks ESE (PR): 25 Marks

Prerequisite: Students should have prior knowledge of

- Knowledge of Python programming and data structures
- Fundamentals of Probability, Statistics, and Linear Algebra
- Basic concepts of Artificial Intelligence and Machine Learning
- Data preprocessing and visualization techniques

Course Objectives: The objective of this course is to provide students with

- To develop the ability to design and implement AI-based search algorithms, such as A* and Greedy search, for solving game, graph, and optimization problems efficiently.
- To acquire skills in building and deploying basic chatbot applications for effective customer interaction using appropriate programming tools and techniques.
- To develop proficiency in preparing, analyzing, and preprocessing diverse real-world datasets, and applying appropriate machine learning techniques for predictive and descriptive analytics.
- To enhance the ability to evaluate machine learning models using relevant performance metrics and visualization techniques, enabling effective interpretation and decision-making in practical applications.

Course Outcomes: After completing this course, students will be able to

- CO1:** To implement and compare AI-based search algorithms, such as A* and Greedy search, to solve game, graph, and optimization problems effectively.
- CO2:** To design, develop, and deploy a basic chatbot application to address specific customer interaction scenarios.
- CO3:** To preprocess, analyze, and apply suitable machine learning models to real-world datasets for both predictive and descriptive analytics.
- CO4:** To assess machine learning model performance using appropriate metrics and visualizations to support data-driven decision-making.

COURSE CONTENTS

Assign. No.	Description of Assignment	CO
1.	Implement A star Algorithm for any game search problem.	CO1
2.	Implement Greedy search algorithm for any of the following application: I. Selection Sort II. Minimum Spanning Tree III. Single-Source Shortest Path Problem IV. Job Scheduling Problem V. Prim's Minimal Spanning Tree Algorithm VI. Kruskal's Minimal Spanning Tree Algorithm VII. Dijkstra's Minimal Spanning Tree Algorithm	CO1
3.	Develop an elementary chat boat for any suitable customer interaction application.	CO2

4.	Data preparation: Download heart dataset from following link. https://www.kaggle.com/zhaoyingzhu/heartcsv Perform following operation on given dataset. a) Find Shape of Data b) Find Missing Values c) Find data type of each column d) Finding out Zero's e) Find Mean age of patients f) Now extract only Age, Sex, ChestPain, RestBP, Chol. Randomly divide dataset in training (75%) and testing (25%).	CO3
5.	Download temperature data from below link. https://www.kaggle.com/venky73/temperatures-of-india?select=temperatures.csv This data consists of temperatures of INDIA averaging the temperatures of all place's month wise. Temperatures values are recorded in CELSIUS a. Apply Linear & Logistic Regression using suitable library function and predict the Month-wise temperature. b. Assess the performance of regression models using MSE, MAE and R-Square metrics c. Visualize simple regression model.	CO3
6.	Assignment on Classification technique Every year many students give the GRE exam to get admission in foreign Universities. The data set contains GRE Scores (out of 340), TOEFL Scores (out of 120), University Rating (out of 5), Statement of Purpose strength (out of 5), Letter of Recommendation strength (out of 5), Undergraduate GPA (out of 10), Research Experience (0=no, 1=yes), Admitted (0=no, 1=yes). Admitted is the target variable. Data Set Available on kaggle (The last column of the dataset needs to be changed to 0 or 1)Data Set : https://www.kaggle.com/mohansacharya/graduate-admissions The counselor of the firm is supposed check whether the student will get an admission or not based on his/her GRE score and Academic Score. So to help the counselor to take appropriate decisions build a machine learning model classifier using Decision tree& Support Vector Machine to predict whether a student will get admission or not. Apply Data pre-processing (Label Encoding, Data Transformation....) techniques if necessary. Perform data-preparation (Train-Test Split) C. Apply Machine Learning Algorithm D. Evaluate Model.	CO3
7.	Download the following customer dataset from below link: Data Set: https://www.kaggle.com/shwetabh123/mall-customers This dataset gives the data of Income and money spent by the customers visiting a shopping mall. The data set contains Customer ID, Gender, Age, Annual Income, Spending Score. Therefore, as a mall owner you need to find the group of people who are the profitable customers for the mall owner. Apply at least two clustering algorithms (based on Spending Score) to find the group of customers. a. Apply Data pre-processing (Label Encoding, Data Transformation....) techniques if necessary. b. Apply Machine Learning Algorithm c. Evaluate Model. d. Visualize clusters.	CO4
8.	Assignment on Association Rule Learning	CO4

	Download Market Basket Optimization dataset from below link. Data Set: https://www.kaggle.com/hemanthkumar05/market-basket-optimization This dataset comprises the list of transactions of a retail company over the period of one week. It contains a total of 7501 transaction records where each record consists of the list of items sold in one transaction. Using this record of transactions and items in each transaction, find the association rules between items. There is no header in the dataset and the first row contains the first transaction, so mentioned header = None here while loading dataset. a. Follow following steps : b. Data Preprocessing c. Generate the list of transactions from the dataset d. Train Apriori algorithm on the dataset e. Visualize the list of rules F. Generated rules depend on the values of hyper parameters. By increasing the minimum confidence value and find the rules accordingly	
Text Books:		
T1:	Stuart Russell and Peter Norvig, <i>Artificial Intelligence: A Modern Approach</i> , Third edition, Pearson, 2003, ISBN :10: 0136042597	
T2:	Ethem Alpaydin, <i>Introduction to Machine Learning</i> , PHI Learning Pvt. Ltd., 4th Edition, 2021, ISBN: 978-9356840225.	
Reference Books:		
R1:	Christopher M. Bishop, <i>Pattern Recognition and Machine Learning</i> , Springer (Special Indian Edition), 2007, ISBN: 978-9380501413.	
R2:	Trevor Hastie et.al, <i>The Elements of Statistical Learning</i> , Springer (Indian Edition), 2nd Edition, 2017, ISBN: 978-8184894753.	
R3:	Aurélien Géron, <i>Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow</i> , Shroff Publishers (India), 3rd Edition, 2022, ISBN: 978-9355421456.	
R4:	Patrick Henry Winston, <i>Artificial Intelligence</i> , Addison-Wesley Publishing Company, ISBN: 0-201-53377-4	
Relevant MOOCs Course (Course name and Weblink):		
1.NPTEL: Machine Learning, Prof. Balaraman Ravindran, IIT Madras https://nptel.ac.in/courses/106106139		
2.NPTEL: Deep Learning, Prof. Mitesh M. Khapra, IIT Madras https://nptel.ac.in/courses/106106201		
3.NPTEL: The Joy of Computing using Python, Prof. Sudarshan Iyengar, IT Ropar https://nptel.ac.in/courses/106106182		
Relevant Topics for Self-study:		
1. Exploring AI’s potential benefits, risks, and impact on employment, privacy, bias, and decision-making in various industries.		
2.Feature Engineering in Machine Learning		

Academic Year :	2026 – 2027	Department :	Info. Tech.
Subject :	Artificial Intelligence and Machine Learning Laboratory (AIMLL)	Class :	T.Y. B. Tech
Subject Code :	2635PC32	Semester :	III

Sr. No.	Title	Performance Date Scheduled	Submission Date Scheduled
01	Implement A star Algorithm for any game search problem.	14/7/2026	28/7/2026
02	Implement Greedy search algorithm for any of the following application: I. Selection Sort II. Minimum Spanning Tree III. Single-Source Shortest Path Problem IV. Job Scheduling Problem V. Prim's Minimal Spanning Tree Algorithm VI. Kruskal's Minimal Spanning Tree Algorithm VII. Dijkstra's Minimal Spanning Tree Algorithm	28/7/26	4/8/2026
03	Develop an elementary chat bot for any suitable customer interaction application.	4/8/2026	11/8/2026
04	Data preparation: Download heart dataset from following link. https://www.kaggle.com/zhaoyingzhu/heartcsv Perform following operation on given dataset. a) Find Shape of Data b) Find Missing Values c) Find data type of each column d) Finding out Zero's e) Find Mean age of patients f) Now extract only Age, Sex, ChestPain, RestBP, Chol. Randomly divide dataset in training (75%) and testing (25%).	11/8/2026	25/08/2026
05	Download temperature data from below link. https://www.kaggle.com/venky73/temperatures-of-india?select=temperatures.csv This data consists of temperatures of INDIA averaging the temperatures of all place's month wise. Temperatures values are recorded in CELSIUS a. Apply Linear & Logistic Regression using suitable library function and predict the Month-wise temperature. b. Assess the performance of regression models using MSE, MAE and R-Square metrics c. Visualize simple regression model.	25/08/2026	08/09/2026
06	Assignment on Classification technique Every year many students give the GRE exam to get admission in foreign Universities. The data set contains GRE Scores (out of 340), TOEFL Scores (out of 120), University Rating (out of 5), Statement of Purpose strength (out of 5), Letter of Recommendation strength (out of 5), Undergraduate GPA (out of	08/09/2026	22/9/2026

	10), Research Experience (0=no, 1=yes), Admitted (0=no, 1=yes). Admitted is the target variable. Data Set Available on kaggle (The last column of the dataset needs to be changed to 0 or 1)Data Set : https://www.kaggle.com/mohansacharya/graduate-admissions The counselor of the firm is supposed check whether the student will get an admission or not based on his/her GRE score and Academic Score. So to help the counselor to take appropriate decisions build a machine learning model classifier using Decision tree& Support Vector Machine to predict whether a student will get admission or not. Apply Data pre-processing (Label Encoding, Data Transformation....) techniques if necessary. Perform data-preparation (Train-Test Split) C. Apply Machine Learning Algorithm D. Evaluate Model.		
07	Download the following customer dataset from below link: Data Set: https://www.kaggle.com/shwetabh123/mall-customers This dataset gives the data of Income and money spent by the customers visiting a shopping mall. The data set contains Customer ID, Gender, Age, Annual Income, Spending Score. Therefore, as a mall owner you need to find the group of people who are the profitable customers for the mall owner. Apply at least two clustering algorithms (based on Spending Score) to find the group of customers. a. Apply Data pre-processing (Label Encoding, Data Transformation....) techniques if necessary. b. Apply Machine Learning Algorithm c. Evaluate Model. d. Visualize clusters.	22/9/2026	06/10/2026
08	Assignment on Association Rule Learning Download Market Basket Optimization dataset from below link. Data Set: https://www.kaggle.com/hemanthkumar05/market-basket-optimization This dataset comprises the list of transactions of a retail company over the period of one week. It contains a total of 7501 transaction records where each record consists of the list of items sold in one transaction. Using this record of transactions and items in each transaction, find the association rules between items. There is no header in the dataset and the first row contains the first transaction, so mentioned header = None here while loading dataset. a. Follow following steps : b. Data Preprocessing c. Generate the list of transactions from the dataset d. Train Apriori algorithm on the dataset e. Visualize the list of rules F. Generated rules depend on the values of hyper parameters. By increasing the minimum confidence value and find the rules accordingly	06/10/2026	27/10/2026

Assignment No -1

Title: Implement A star Algorithm for any game search problem.

Problem Statement:

Implement A star Algorithm for any game search problem.

Objective: This assignment will help the students develop the ability to design and implement AI-based search algorithms, such as A* and Greedy search, for solving game, graph, and optimization problems efficiently.

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows , Jupyter notebook.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15''Color Monitor, Keyboard, Mouse

References:

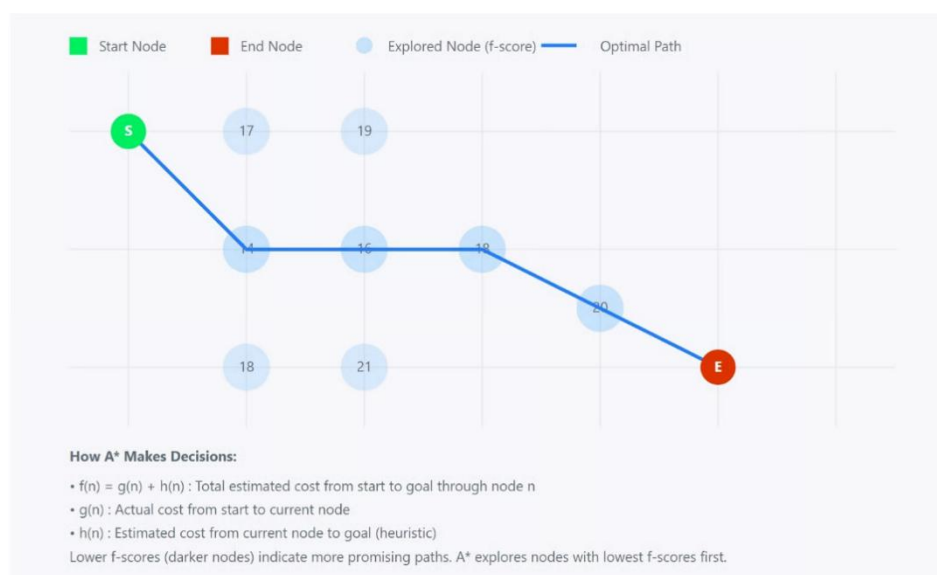
Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Third edition, Pearson, 2003, ISBN :10: 0136042597.

Theory:

A* STAR Algorithm

The A* algorithm is an informed search algorithm, meaning it leverages a heuristic function to guide its search towards the goal. This heuristic function estimates the cost of reaching the goal from a given node, allowing the algorithm to prioritize promising paths and avoid exploring unnecessary ones.

The A* algorithm is a powerful and widely used graph traversal and path finding algorithm. It finds the shortest path between a starting node and a goal node in a weighted graph.



How Does the A* Algorithm Work?

The A* algorithm combines the best aspects of two other algorithms:

1. **Dijkstra's Algorithm:** This algorithm finds the shortest path to all nodes from a single source node.
2. **Greedy Best-First Search:** This algorithm explores the node that appears to be closest to the goal, based on a heuristic function.

While Dijkstra's algorithm would explore in all directions and Best-First Search might head straight toward the destination (potentially missing shortcuts), A* does something cleverer. It considers both:

- The distance already traveled from the start
- A smart estimate of the remaining distance to the goal

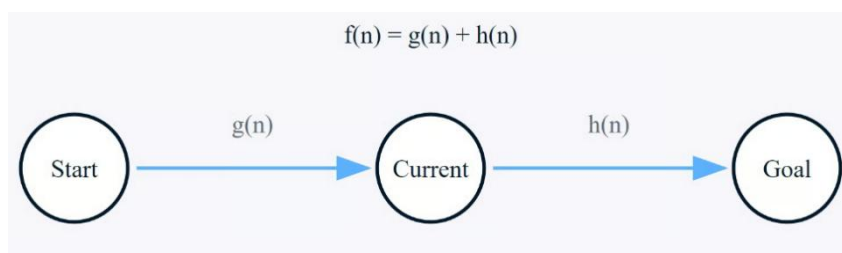
Key components:

To understand A* algorithm, you need to be familiar with these fundamental concepts:

- **Nodes:** Points in your graph (like intersections on a map)
- **Edges:** Connections between nodes (like roads connecting intersections)
- **Path Cost:** The actual cost of moving from one node to another
- **Heuristic:** An estimated cost from any node to the goal
- **Search Space:** The collection of all possible paths to explore

Key Concepts in A* Search:

The A* algorithm's efficiency comes from its smart evaluation of paths using three key components: $g(n)$, $h(n)$, and $f(n)$. These components work together to guide the search process toward the most promising paths.



Understanding the cost functions

Path cost $g(n)$

The path cost function $g(n)$ represents the exact, known distance from the initial starting node to the current position in our search. Unlike estimated values, this cost is precise and calculated by adding up all the individual edge weights that have been traversed along our chosen path.

Mathematically, for a path through nodes n_0 (start node) to n_k (current node), we can express $g(n)$ as:

$$g(n_k) = \sum_{i=0}^{k-1} w(n_i, n_{i+1})$$

Where:

- $w(n_i, n_{i+1})$ represents the weight of the edge connecting node n_i to node n_{i+1} .

As we move through the graph, this value accumulates, giving us a clear measure of the actual resources (whether that's distance, time, or any other metric) we've expended to reach our current position.

Heuristic function $h(n)$

The heuristic function $h(n)$ provides an estimated cost from the current node to the goal node, acting as the algorithm's "informed guess" about the remaining path.

Mathematically, for any given node n , the heuristic estimate must satisfy the condition $h(n) \leq h^*(n)$, where $h^*(n)$ is the actual cost to the goal, making it admissible by never overestimating the true cost.

In grid-based or map-like problems, common heuristic functions include the [Manhattan distance](#) and [Euclidean distance](#). For coordinates (x_1, y_1) of the current node and (x_2, y_2) of the goal node, these distances are calculated as:

Manhattan distance

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

Euclidean distance

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Total estimated cost $f(n)$

The total estimated cost $f(n)$ is the cornerstone of A* algorithm's decision-making process, combining both the actual path cost and the heuristic estimate to evaluate each node's potential. For any node n , this cost is calculated as:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ represents the actual cost from the start to the current node,
- $h(n)$ represents the estimated cost from the current node to the goal.

The [algorithm](#) uses this combined value to strategically choose which node to explore next, always selecting the node with the lowest $f(n)$ value from the open list, thus ensuring an optimal balance between known costs and estimated remaining distances.

Managing node lists

The A* algorithm maintains two essential lists

Open list:

- Contains nodes that need to be evaluated
- Sorted by $f(n)$ value (lowest first)
- New nodes are added as they're discovered

Closed list:

- Contains already evaluated nodes
- Helps avoid re-evaluating nodes

- Used to reconstruct the final path

The algorithm continually selects the node with the lowest $f(n)$ value from the open list, evaluates it, and moves it to the closed list until it reaches the goal node or determines no path exists.

A* Search Algorithm Pseudocode

```
function A_Star(start, goal):
    // Initialize open and closed lists
    openList = [start]      // Nodes to be evaluated
    closedList = []         // Nodes already evaluated

    // Initialize node properties
    start.g = 0             // Cost from start to start is 0
    start.h = heuristic(start, goal) // Estimate to goal
    start.f = start.g + start.h // Total estimated cost
    start.parent = null     // For path reconstruction
    while openList is not empty:
        // Get node with lowest f value - implement using a priority queue
        // for faster retrieval of the best node
        current = node in openList with lowest f value

        // Check if we've reached the goal
        if current = goal:
            return reconstruct_path(current)

        // Move current node from open to closed list
        remove current from openList
        add current to closedList

        // Check all neighboring nodes
        for each neighbor of current:
            if neighbor in closedList:
                continue // Skip already evaluated nodes

            // Calculate tentative g score
            tentative_g = current.g + distance(current, neighbor)
```

```
    if neighbor not in openList:
        add neighbor to openList
    else if tentative_g >= neighbor.g:
        continue // This path is not better

    // This path is the best so far
    neighbor.parent = current
    neighbor.g = tentative_g
    neighbor.h = heuristic(neighbor, goal)
    neighbor.f = neighbor.g + neighbor.h

return failure // No path exists

function reconstruct_path(current):
    path = []
    while current is not null:
        add current to beginning of path
        current = current.parent
    return path
```

Applications of A* Search Algorithm

The A* algorithm's efficiency and flexibility make it valuable across numerous domains. Here are the key areas where it excels:

1. Video Games and entertainment

The A* search algorithm is extensively used in video game development due to its optimal pathfinding capabilities. It enhances the player experience by allowing for more realistic and responsive character movement.

- Character pathfinding in strategy games: A* helps characters find the shortest or safest path to a target, crucial in real-time strategy (RTS) games where units need to navigate around obstacles and enemies effectively.
- NPC movement in open-world environments: Non-playable characters (NPCs) use A* to navigate large and dynamic game worlds, allowing them to reach objectives while avoiding obstacles and other characters.
- Real-time tactical planning in combat scenarios: A* is used to calculate the efficient movement and positioning of units during combat, ensuring that characters can quickly reach advantageous spots while avoiding danger.
- Maze solving in puzzle games: A* is an effective algorithm for navigating complex mazes, giving players an engaging challenge by powering dynamic maze-solving opponents or providing hints.

2. Navigation systems

A* is widely used in navigation systems to optimize routes, accounting for various factors like distance and potential obstacles.

- Route planning in GPS applications: A* finds the shortest and fastest route between two points, making it an essential component of GPS software used by millions worldwide.
- Traffic-aware navigation services: In modern navigation apps, A* is combined with real-time traffic data to suggest the best route, minimizing travel time and avoiding congested roads.
- Public transport route optimization: A* can help find optimal routes that incorporate multiple modes of public transport, ensuring users make efficient transfers.
- Indoor navigation systems: A* is also used in indoor navigation, such as in airports or large shopping malls, where it helps guide users through complex environments with multiple levels and obstacles.

3. Robotics and automation

The A* algorithm is crucial for robotics, where efficient movement is essential for productivity and safety.

- Autonomous vehicle path planning: Self-driving cars use A* to navigate roads, making decisions in real time about how to move from point A to point B while avoiding collisions and adhering to traffic rules.
- Warehouse robot navigation: In automated warehouses, robots rely on A* to navigate efficiently between storage racks to pick and place items, minimizing delays and avoiding collisions with other robots.
- Drone flight path optimization: A* helps drones plan efficient flight paths, whether for delivery, surveying, or recreational use, ensuring they avoid obstacles and follow optimal routes.
- Manufacturing robot movement planning: In factory settings, A* is used to ensure robots can move seamlessly between workstations, avoiding collisions with other machinery and maintaining productivity.

4. Network systems

A* is also applied in optimizing network operations, where efficiency in resource utilization and routing is paramount.

- Network packet routing: A* is used to determine the most efficient path for data packets to travel across a network, ensuring minimal latency and high reliability.
- Resource allocation in distributed systems: A* aids in optimizing resource allocation, enabling distributed systems to efficiently allocate tasks across various nodes while minimizing overhead.
- Circuit board path design: When designing printed circuit boards (PCBs), A* can be used to determine optimal paths for electrical traces, ensuring minimal interference and efficient layout.
- Network cable routing optimization: A* is employed in designing physical network infrastructures, finding the most effective routes for cables to minimize cost and maximize performance.

What makes A* particularly valuable is its adaptability through custom heuristic functions, allowing optimization for different metrics like distance, time, or energy usage.

In the next section, we'll look at some common challenges and optimization techniques for implementing A* effectively.

Common Challenges and Optimization Techniques

While A* is powerful, implementing it effectively requires addressing several common challenges. The most significant hurdle developers face is managing resources efficiently, particularly when dealing with large search spaces.

The primary challenges include:

- Memory consumption in large graphs
- Performance bottlenecks with complex heuristics
- Handling tie-breaking situations
- Balancing accuracy vs. computation speed

Fortunately, there are several effective optimization strategies to address these challenges:

- For memory management, focus on efficient [data structures](#)
- Use binary heaps for the open list instead of arrays
- Implement a hash table for faster closed list lookups
- Clear unnecessary node data after processing

When performance is critical, consider these speed improvements:

- Simplify heuristic calculations where possible
- Use integer arithmetic instead of floating-point
- Implement hierarchical pathfinding for large maps

One particularly effective approach for large spaces is bilateral search—searching from both the start and goal simultaneously. Additionally, when working with grid-based pathfinding, you can significantly improve performance by pre-computing heuristic values or using lookup tables.

Remember to choose optimization techniques based on your specific requirements and constraints. The key is finding the right balance between memory usage and computational speed for your application.

Conclusion

The A* algorithm stands as a fundamental tool in pathfinding and graph traversal problems. Through this guide, we have seen its core concepts, implemented a practical solution in Python, and examined its diverse applications. The algorithm's strength lies in its balance of accuracy and efficiency, making it invaluable across various domains from gaming to robotics.

Implementation of A* Algorithm for 8-Puzzle Game

Aim

Implement the A* Algorithm to solve the 8-Puzzle game.

Problem Statement

The 8-puzzle consists of eight numbered tiles and one blank space arranged in a 3×3 grid. The objective is to reach the goal state by sliding the tiles into the blank space using the A* search algorithm.

Algorithm

Initialize the start state and goal state.

Calculate the heuristic value (Manhattan Distance).

Store the initial state in the priority queue (Open List).

Repeat until the goal is found: remove the state with the minimum $f(n)=g(n)+h(n)$. If it is the goal state, stop. Otherwise generate valid neighboring states, compute g, h and f, and add unexplored states to the priority queue.

Print the solution path.

Python Implementation

```
from queue import PriorityQueue
```

```
goal = [[1,2,3],[4,5,6],[7,8,0]]
```

class Node:

```
def __init__(self, state, parent=None, g=0):
```

```
    self.state = state
```

```
    self.parent = parent
```

```
    self.g = g
```

```
    self.h = self.heuristic()
```

```
    self.f = self.g + self.h
```

```
def heuristic(self):
```

```
    distance = 0
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            value = self.state[i][j]
```

```
            if value != 0:
```

```
                goal_x = (value - 1) // 3
```

```
                goal_y = (value - 1) % 3
```

```
                distance += abs(i - goal_x) + abs(j - goal_y)
```

```
    return distance
```

```
def __lt__(self, other):
```

```
    return self.f < other.f
```

```
def get_neighbors(state):
```

```
    neighbors = []
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if state[i][j] == 0:
```

```
                x, y = i, j
```

```
    moves = [(-1,0),(1,0),(0,-1),(0,1)]
```

```
    for dx, dy in moves:
```

```
        nx, ny = x+dx, y+dy
```

```
        if 0 <= nx < 3 and 0 <= ny < 3:
```



```

        new_state = [row[:] for row in state]
        new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
        neighbors.append(new_state)
    return neighbors

def state_to_tuple(state):
    return tuple(tuple(row) for row in state)

def print_path(node):
    path=[]
    while node:
        path.append(node.state)
        node=node.parent
    path.reverse()
    for step in path:
        for row in step:
            print(row)
        print()

def astar(start):
    open_list=PriorityQueue()
    visited=set()
    open_list.put(Node(start))
    while not open_list.empty():
        current=open_list.get()
        if current.state==goal:
            print("Goal Reached!")
            print("Total Moves:", current.g)
            print_path(current)
            return
        visited.add(state_to_tuple(current.state))
        for neighbor in get_neighbors(current.state):
            if state_to_tuple(neighbor) not in visited:
                open_list.put(Node(neighbor,current,current.g+1))
    print("No Solution Exists")

start=[[1,2,3],[4,0,6],[7,5,8]]
astar(start)

```

Sample Output

Goal Reached!
Total Moves: 2

Solution Path:

[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]

Time Complexity

Worst Case: $O(b^d)$, where b is the branching factor and d is the depth of the optimal solution.

Space Complexity $O(b^d)$

Applications

- 8-Puzzle and 15-Puzzle games
- Pathfinding in maps and GPS navigation
- Robot motion planning
- AI game playing
- Network routing
- Maze solving.

Assignment No -2

Title: Implement Greedy search algorithm

Problem Statement:

Implement Greedy search algorithm for any of the following applications:

- I. Selection Sort
- II. Minimum Spanning Tree
- III. Single-Source Shortest Path Problem
- IV. Job Scheduling Problem
- V. Prim's Minimal Spanning Tree Algorithm
- VI. Kruskal's Minimal Spanning Tree Algorithm
- VII. Dijkstra's Minimal Spanning Tree Algorithm

Objective:

This assignment will help the students develop the ability to design and implement AI-based search algorithms, such as A* and Greedy search, for solving game, graph, and optimization problems efficiently.

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows , Jupyter notebook.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15''Color Monitor, Keyboard, Mouse

References:

Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Third edition, Pearson, 2003, ISBN :10: 0136042597.

Theory:

The **Single-Source Shortest Path (SSSP)** problem aims to find the shortest path from one source vertex to every other vertex in a weighted graph.

Dijkstra's Algorithm is a greedy algorithm that computes the shortest path from a source vertex to all other vertices in a graph with **non-negative edge weights**.

The algorithm repeatedly selects the vertex with the minimum tentative distance and updates the distances of its adjacent vertices.

Algorithm

1. Initialize the distance of the source vertex as **0** and all other vertices as ∞ (**infinity**).
2. Mark all vertices as unvisited.
3. Select the unvisited vertex with the smallest distance.
4. Mark the selected vertex as visited.

5. Update the distances of all adjacent vertices if a shorter path is found.
6. Repeat Steps 3–5 until all vertices are visited.
7. Display the shortest distance from the source to every vertex.

Time Complexity

Implementation	Time Complexity
Using Priority Queue (Heap)	$O((V + E) \log V)$
Using Adjacency Matrix	$O(V^2)$

```
import heapq

def dijkstra(graph, start):

    distances = {vertex: float('inf') for vertex in graph}
    distances[start] = 0

    priority_queue = [(0, start)]

    while priority_queue:
        current_distance, current_vertex =
heapq.heappop(priority_queue)

        if current_distance > distances[current_vertex]:
            continue

        for neighbor, weight in graph[current_vertex]:

            distance = current_distance + weight

            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(priority_queue, (distance,
neighbor))

    return distances

# Graph Representation
graph = {
    'A': [('B', 4), ('C', 2)],
    'B': [('A', 4), ('C', 1), ('D', 5)],
    'C': [('A', 2), ('B', 1), ('D', 8), ('E', 10)],
    'D': [('B', 5), ('C', 8), ('E', 2), ('F', 6)],
    'E': [('C', 10), ('D', 2), ('F', 3)],
    'F': [('D', 6), ('E', 3)]
}
source = 'A'
shortest_paths = dijkstra(graph, source)

print("Shortest distances from Source Vertex:", source)

for vertex, distance in shortest_paths.items():
    print(f"{source} -> {vertex} = {distance}")
```

Prim's Minimum Spanning Tree (MST) Algorithm

Aim

To implement **Prim's Minimum Spanning Tree (MST) Algorithm** to find the minimum spanning tree of a connected weighted graph.

Theory

A **Minimum Spanning Tree (MST)** is a subset of edges in a connected, weighted, undirected graph that:

- Connects all vertices.
- Contains no cycles.
- Has the minimum possible total edge weight.

Prim's Algorithm is a greedy algorithm that starts from any vertex and repeatedly selects the minimum-weight edge that connects a visited vertex to an unvisited vertex until all vertices are included.

Algorithm

1. Select any vertex as the starting vertex.
2. Mark the starting vertex as visited.
3. Find the minimum-weight edge connecting a visited vertex to an unvisited vertex.
4. Add the selected edge to the MST.
5. Mark the new vertex as visited.
6. Repeat Steps 3–5 until all vertices are visited.
7. Display the edges of the MST and its total cost.

Time Complexity

- Using Priority Queue: $O(E \log V)$

Program

```
import heapq

def prim(graph, start):
    visited = set([start])
    edges = []

    for neighbor, weight in graph[start]:
        heapq.heappush(edges, (weight, start, neighbor))

    mst = []
    total_cost = 0

    while edges:
        weight, u, v = heapq.heappop(edges)

        if v not in visited:
            visited.add(v)
            mst.append((u, v, weight))
            total_cost += weight

            for neighbor, w in graph[v]:
                if neighbor not in visited:
                    heapq.heappush(edges, (w, v, neighbor))

    return mst, total_cost

graph = {
    'A': [('B', 2), ('C', 3)],
    'B': [('A', 2), ('C', 1), ('D', 1)],
    'C': [('A', 3), ('B', 1), ('D', 4), ('E', 5)],
    'D': [('B', 1), ('C', 4), ('E', 2)],
    'E': [('C', 5), ('D', 2)]
}

mst, cost = prim(graph, 'A')

print("Edges in MST:")
for u, v, w in mst:
    print(f"{u} - {v} : {w}")

print("Total Cost =", cost)
```

Kruskal's Minimum Spanning Tree (MST) Algorithm

Aim

To implement **Kruskal's Minimum Spanning Tree Algorithm** for finding the minimum spanning tree of a connected weighted graph.

Theory

Kruskal's Algorithm is a greedy algorithm that builds the Minimum Spanning Tree by repeatedly selecting the edge with the smallest weight without forming a cycle.

It uses the **Disjoint Set (Union-Find)** data structure to efficiently detect cycles.

Algorithm

1. Sort all graph edges in increasing order of weight.
2. Initialize each vertex as a separate set.
3. Select the smallest edge.
4. If adding the edge does not create a cycle:
 - Add it to the MST.
 - Union the two sets.
5. Repeat until $V - 1$ edges are selected.
6. Display the MST and its total cost.

Time Complexity

- Sorting Edges: $O(E \log E)$
- Union-Find Operations: $O(E \alpha(V))$ (approximately linear)

Overall Complexity:

$O(E \log E)$

```
class DisjointSet:

    def __init__(self, vertices):
        self.parent = {v: v for v in vertices}

    def find(self, item):
        if self.parent[item] != item:
            self.parent[item] = self.find(self.parent[item])
        return self.parent[item]

    def union(self, x, y):
        rootX = self.find(x)
        rootY = self.find(y)

        if rootX != rootY:
            self.parent[rootY] = rootX

def kruskal(vertices, edges):

    edges = sorted(edges, key=lambda x: x[2])

    ds = DisjointSet(vertices)

    mst = []
    total_cost = 0

    for u, v, weight in edges:

        if ds.find(u) != ds.find(v):
            ds.union(u, v)
            mst.append((u, v, weight))
            total_cost += weight

    return mst, total_cost

vertices = ['A', 'B', 'C', 'D', 'E']

edges = [
    ('A', 'B', 2),
    ('A', 'C', 3),
    ('B', 'C', 1),
    ('B', 'D', 1),
    ('C', 'D', 4),
    ('C', 'E', 5),
    ('D', 'E', 2)
]

mst, cost = kruskal(vertices, edges)

print("Edges in MST:")
for u, v, w in mst:
    print(f"{u} - {v} : {w}")

print("Total Cost =", cost)
```


Assignment No -3

Title: Develop an Elementary Chatbot for a Suitable Customer Interaction Application

Problem Statement:

Develop a simple chatbot for a customer support application (e.g., College Admission, Online Shopping, Banking, Hospital, or Restaurant). The chatbot should answer predefined customer queries using rule-based logic.

Objective:

This assignment will help the students to design, develop, and deploy a basic chatbot application to address specific customer interaction scenarios.

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows , Jupyter notebook.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15''Color Monitor, Keyboard, Mouse

References:

Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Third edition, Pearson, 2003, ISBN :10: 0136042597.

Theory:

A chatbot is a software application that simulates human conversation through text or voice. Chatbots are widely used in customer service to provide instant responses, reduce workload, and improve customer satisfaction.

Types of Chatbots

1. Rule-Based Chatbots

- Respond using predefined rules and keywords.
- Easy to develop.
- Limited to predefined conversations.

2. AI-Based Chatbots

- Use Natural Language Processing (NLP) and Machine Learning.
- Learn from user interactions.
- Handle complex conversations.

Algorithm

1. Start the program.

2. Display a welcome message.
3. Accept the user's query.
4. Convert the input to lowercase.
5. Match the query with predefined keywords.
6. Display the corresponding response.
7. If no keyword matches, display a default response.
8. Repeat until the user enters **exit** or **bye**.
9. Stop the program.

```
print("=====")
print(" Welcome to ABC Customer Support ")
print(" Type 'exit' to end the chat.")
print("=====")
```

```
while True:
```

```
    user = input("You: ").lower()
```

```
    if user == "exit" or user == "bye":
```

```
        print("Bot: Thank you for visiting. Have a nice day!")
```

```
        break
```

```
    elif "hello" in user or "hi" in user:
```

```
        print("Bot: Hello! How can I assist you today?")
```

```
    elif "admission" in user:
```

```
        print("Bot: Admissions are open. Please visit the admission office or our website.")
```

```
    elif "fees" in user:
```

```
        print("Bot: The fee details are available on the college website.")
```

```
    elif "courses" in user:
```

```
        print("Bot: We offer B.E., M.E., MCA and Ph.D. programs.")
```

```
    elif "contact" in user:
```

```
        print("Bot: You can contact us at 020-12345678.")
```

```
elif "thank" in user:
```

```
    print("Bot: You're welcome!")
```

```
else:
```

```
    print("Bot: Sorry, I didn't understand your question.")
```

Sample Output:

Welcome to ABC Customer Support Type 'exit' to end the chat.

You: Hello Bot: Hello! How can I assist you today?

You: What courses are available?

Bot: We offer B.E., M.E., MCA and Ph.D. programs.

You: Tell me about fees.

Bot: The fee details are available on the college website.

You: Thank you Bot: You're welcome!

You: exit Bot: Thank you for visiting. Have a nice day!

Applications

- College admission helpdesk
- Online shopping support
- Banking customer service
- Hospital appointment assistance
- Restaurant ordering system
- Railway inquiry system

Advantages

- Provides instant responses.
- Available 24×7.
- Reduces customer support workload.
- Easy to develop and maintain.
- Improves customer experience.

Limitations

- Cannot understand complex queries.
- Limited to predefined responses.

- Does not learn automatically.
- Requires manual updates.

Viva Questions

1. What is a chatbot?
2. What is the difference between a rule-based chatbot and an AI chatbot?
3. What is Natural Language Processing (NLP)?
4. Why is lower() used in the chatbot program?
5. What is keyword matching?
6. What are the applications of chatbots?
7. What are the limitations of rule-based chatbots?
8. How can machine learning improve chatbot performance?
9. What happens when no keyword matches the user's query?
10. Name two Python libraries used for AI chatbot development.

Assignment No -4

Title: Data preparation

Problem Statement:

Perform following operation on given dataset:

- a) Find Shape of Data
- b) Find Missing Values
- c) Find data type of each column
- d) Finding out Zero's
- e) Find Mean age of patients
- f) Now extract only Age, Sex, ChestPain, RestBP, Chol. Randomly divide dataset in training (75%) and testing (25%).

Objective: This assignment will help the students to realize what is needed of data preparation

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows , Jupyter notebook.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15''Color Monitor, Keyboard, Mouse

References:

1. Ethem Alpaydin, Introduction to Machine Learning, PHI 2nd Edition,2013.
2. Peter Flach: Machine Learning: The Art and Science of Algorithms that Make Sense of Data,Cambridge University Press, Edition 2012.

Theory:

Data Preparation

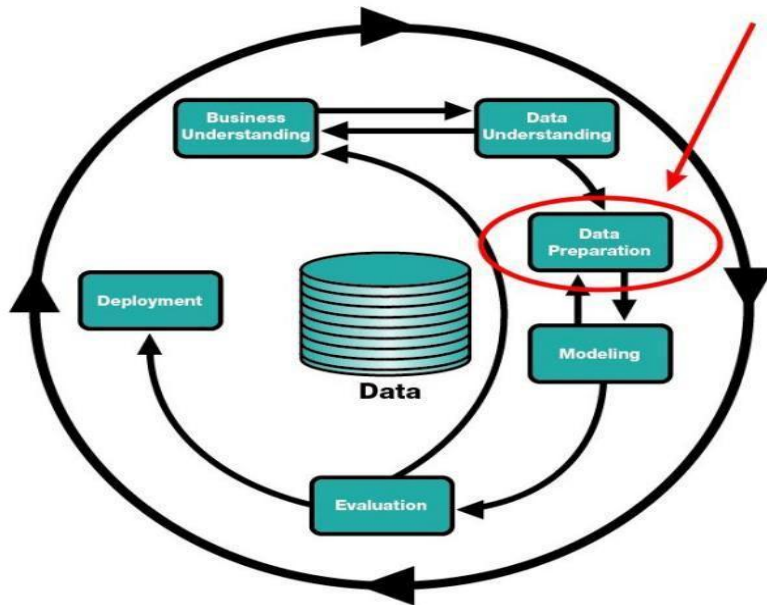
Data preparation (also referred to as “data preprocessing”) is the process of transforming raw data so that data scientists and analysts can run it through machine learning algorithms to uncover insights or make predictions.

Why is Data Preparation Important?

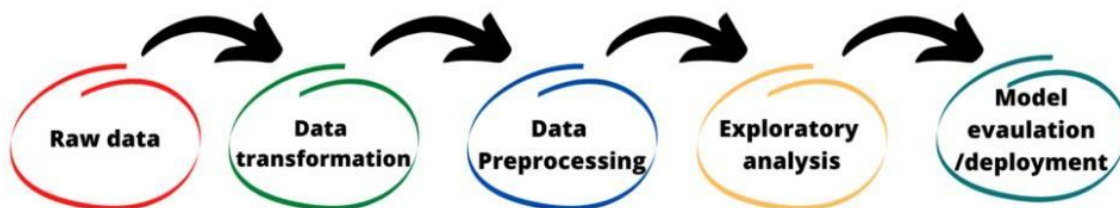
Most machine learning algorithms require data to be formatted in a very specific way, so datasets generally require some amount of preparation before they can yield useful insights. Some datasets have values that are missing, invalid, or otherwise difficult for an algorithm to process. If data is missing, the algorithm can't use it. If data is invalid, the algorithm produces less accurate or even misleading outcomes. Some datasets are relatively clean but need to be shaped (e.g., aggregated or pivoted) and many datasets are just lacking useful business context (e.g., poorly defined ID values), hence the need for feature enrichment. Good data preparation produces clean and well-curated data which leads to more practical, accurate model outcomes.

It is the most required process before feeding the data into the machine learning model. The reason behind that the data set needs to be different and specific according to the model so that

we have to find out the required features of that data. The data preparation process offers a method via which we can prepare the data for defining the project and also for the project evaluation of ML algorithms. Different many predicting machine learning models are there with a different process but some of the processes are common that are performed in every model, and also it allows us to find out the actual business problem and their solutions. Some of the data preparation processes are:



1. Determine the problems
2. Data cleaning
3. Feature selection
4. Data transformation
5. feature engineering
6. Dimensionality reduction



1. Determine the problems:

This step tells us about the learning method of the project to find out the results for future prediction or forecasting. For example, which ML model suitable for the data set regression or

classification or clustering algorithms. This includes data collection that is useful for predicting the result and also involving the communication to project stakeholders and domain expertise. We use classification and regression models for categorical and numerical data respectively.

It includes determining the relevant attributes with the stied data in form of .csv, .html, .json, .doc, and many, also for unstructured data in a form for audio, video, text, images, etc for scanning and detect the patterns of data with searching and identifying the data that have taken from external repositories.

2. Data cleaning:

After collecting the data, it is very necessary to clean that data and make it proper for the ML model. It includes solving problems like outliers, inconsistency, missing values, incorrect, skewed, and trends. Cleaning the data is very important as the model learning from that data only, so if we feed inconsistent, appropriate data to model it will return garbage only, so it is required to make sure that the data does not contains any unseen problem. For example, if we have a data set of sales, it might be possible that it contains some features like height, age, that cannot help in the model building so we can remove it. We generally remove the null values columns, fill the missing values, make the data set consistent, and remove the outliers and skewed data in data cleaning.

3. Feature selection:

Sometimes we face the problem of identifying the related features from the set of data and deleting the irrelevant and less important data without touching the target variables to get the better accuracy of the model. Features selection plays a wide role in building a machine learning model that impacts the performance and accuracy of the model. It is that process which contributes mostly to the predictions or output that we need by selecting the features automatically or manually. If we have irrelevant data that would cause the model with overfitting and underfitting.

The benefits of feature selection:

1. Reduce the overfitting/underfitting
2. Improves the accuracy
3. Reduced training/testing time
4. Improve performance

4. Data transformation:

Data transformation is the process that converts the data from one form to another. It is required for data integration and data management. In data transformation, we can change the types of data, clear the data removing the null values or duplicate values, and enrich data that depends on the requirements of the model. It allows us to perform data mapping that determines how individual features are mapped, modified, filtered, aggregated, and joined. Data transformation is needed for both structured and unstructured data but it is time consuming, costly, slow.

5. Feature engineering:

Every ML algorithms use some input data for giving required output and this input required some features which are in a structured form. To get the proper result the algorithms required features with some specific characteristics which we find out with feature engineering. we need to perform different feature engineering on different datasets and we can observe their effect on model performance. Here I am listing out the techniques of feature engineering.

1. Imputation
2. Handling outliers
3. Binning
4. Log transform
5. one-hot encoding
6. Grouping operations
7. Feature split
8. Scaling

6. Dimensionality reduction:

When we use the dataset for building an ML model, we need to work with 1000s of features that cause the curse of dimensionality, or we can say that it refers to the process to convert a set of data. For the ML model, we have to access a large amount of data and that large amount of data can lead us in a situation where we can take possible data that can be available to feed it into a forecasting model to predict and give the result of the target variable. It reduced the time that is required for training and testing our machine learning model and also helps to eliminate over-fitting. It is kind of zipping the data for the model.

Implementation:

```
import numpy as nm
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("Heart.csv")
```

```
df.head(8)
```


	Unna med: 0	A g e	S e x	ChestP ain	Re st B P	C h o l	Fbs	Rest EC G	M ax H R	Ex An g	O ld p e a k	Slope	C a	Thal	AHD
0	1	6 3	1	typical	14 5	23 3	1	2	15 0	0	2. 3	3	0. 0	fixed	No
1	2	6 7	1	asympto matic	16 0	28 6	0	2	10 8	1	1. 5	2	3. 0	normal	Yes
2	3	6 7	1	asympto matic	12 0	22 9	0	2	12 9	1	2. 6	2	2. 0	reversa ble	Yes
3	4	3 7	1	nonangi nal	13 0	25 0	0	0	18 7	0	3. 5	3	0. 0	normal	No
4	5	4 1	0	nontypi cal	13 0	20 4	0	2	17 2	0	1. 4	1	0. 0	normal	No
5	6	5 6	1	nontypi cal	12 0	23 6	0	0	17 8	0	0. 8	1	0. 0	normal	No
6	7	6 2	0	asympto matic	14 0	26 8	0	2	16 0	0	3. 6	3	2. 0	normal	Yes
7	8	5 7	0	asympto matic	12 0	35 4	0	0	16 3	1	0. 6	1	0. 0	normal	No

In []:

```
df.shape
```

```
(303, 15)
```

```
df.isnull().sum()
```

Out []:

```
Unnamed: 0    0
```

```
Age          0
```

```
Sex          0
```

```
ChestPain    0
```

```
RestBP       0
```

```
Chol      0
Fbs       0
RestECG   0
MaxHR     0
ExAng     0
Oldpeak   0
Slope     0
Ca        4
Thal      2
AHD       0
dtype: int64
```

In []:

```
print("Total missing values: ", df.isnull().sum().sum())
```

Total missing values: 6

In []:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 303 entries, 0 to 302
```

```
Data columns (total 15 columns):
```

```
#   Column      Non-Null Count  Dtype
---  ---
0   Unnamed: 0    303 non-null   int64
```

```
1   Age          303 non-null   int64
```

```
2   Sex          303 non-null   int64
```

```
3   ChestPain    303 non-null   object
```

```
4   RestBP       303 non-null   int64
```

```
5   Chol         303 non-null   int64
```

```
6   Fbs          303 non-null   int64
```

```
7   RestECG      303 non-null   int64
```

```
8   MaxHR        303 non-null   int64
```

```
9   ExAng        303 non-null   int64
```

```
10  Oldpeak      303 non-null   float64
```

```
11  Slope        303 non-null   int64
```

```
12  Ca           299 non-null   float64
```

```
13  Thal         301 non-null   object
```

```
14  AHD          303 non-null   object
```

```
dtypes: float64(2), int64(10), object(3)
```

```
memory usage: 32.0+ KB
```

```
In [ ]:
```

```
df.dtypes
```

```
Out[ ]:
```

```
Unnamed: 0    int64
```

```
Age          int64
```

```
Sex          int64
```

```
ChestPain    object
```

```
RestBP       int64
```

```
Chol         int64
```

```
Fbs         int64
```

```
RestECG      int64
```

```
MaxHR        int64
```

```
ExAng        int64
```

```
Oldpeak      float64
```

```
Slope        int64
```

```
Ca           float64
```

```
Thal         object
```

```
AHD          object
```

```
dtype: object
```

```
In [ ]:
```

```
(df == 0).sum(axis=0)
```

```
Out[ ]:
```

```
Unnamed: 0    0
```

```
Age          0
```

```
Sex          97
```

```
ChestPain    0
```

```
RestBP       0
```

```
Chol         0
```

```
Fbs         258
```

```
RestECG      151
```

```
MaxHR        0
```

```
ExAng        204
```

```
Oldpeak      99
```

```
Slope        0
```

```
Ca      176
Thal     0
AHD      0
dtype: int64
```

In []:

```
mean_age = df['Age'].mean()
mean_age
```

Out[]:

```
54.43894389438944
```

In []:

```
df.columns
```

Out[]:

```
Index(['Unnamed: 0', 'Age', 'Sex', 'ChestPain', 'RestBP', 'Chol', 'Fbs',
      'RestECG', 'MaxHR', 'ExAng', 'Oldpeak', 'Slope', 'Ca', 'Thal', 'AHD'],
      dtype='object')
```

In []:

```
df2 = df.filter(['Age', 'Sex', 'ChestPain', 'RestBP', 'Chol'])
```

In []:

```
df2
```

Out[]:

	Age	Sex	ChestPain	RestBP	Chol
0	63	1	typical	145	233
1	67	1	asymptomatic	160	286
2	67	1	asymptomatic	120	229
3	37	1	nonanginal	130	250
4	41	0	nontypical	130	204
...	...	...	...	...	...
29 8	45	1	typical	110	264

299	68	1	asymptomatic	144	193
300	57	1	asymptomatic	130	131
301	57	0	nontypical	130	236
302	38	1	nonanginal	138	175

303 rows × 5 columns

```
mean = df['Ca'].mean()
df['Ca'].fillna(value=mean,inplace=True)
```

```
mode = df['Thal'].mode().iloc[0]
df['Thal'].fillna(value=mode, inplace=True)
```

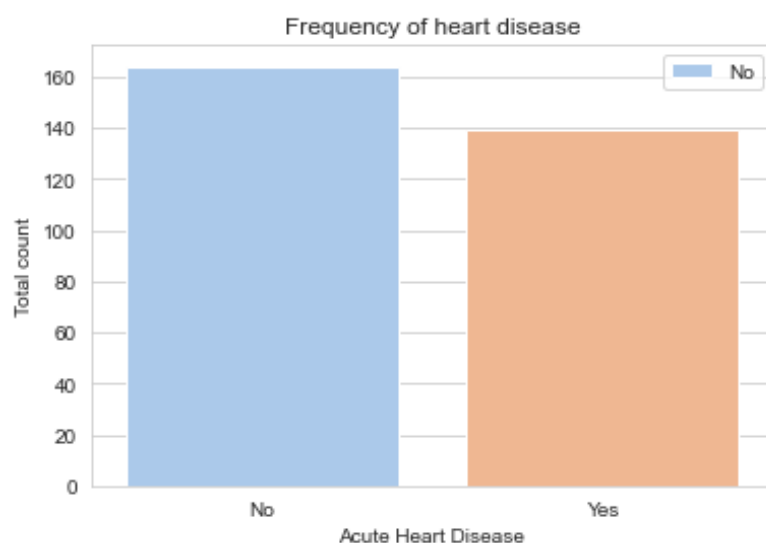
```
df.isnull().sum()
```

```
Unnamed: 0    0
Age           0
Sex           0
ChestPain     0
RestBP        0
Chol          0
Fbs           0
RestECG       0
MaxHR         0
ExAng         0
Oldpeak       0
Slope         0
Ca            0
Thal          0
AHD           0
dtype: int64
```

```
sns.countplot(x='AHD',data=df, palette='pastel')
sns.set_style("whitegrid")

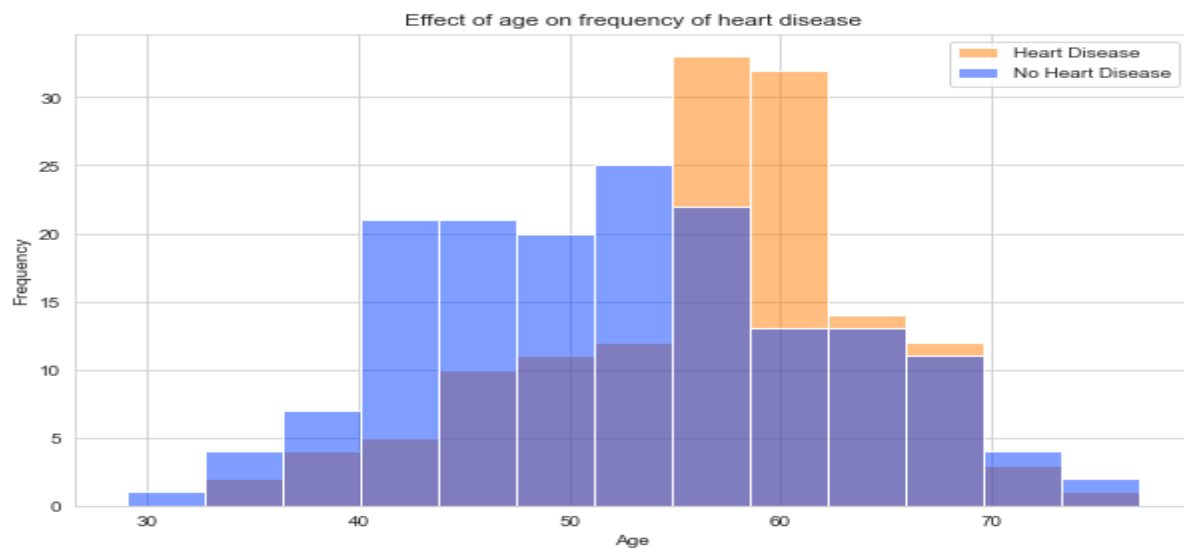
plt.xlabel("Acute Heart Disease")
plt.ylabel("Total count")
plt.title("Frequency of heart disease")
plt.legend(['No','Yes'],loc='upper right')
```

<matplotlib.legend.Legend at 0x6d758f8>



In []:

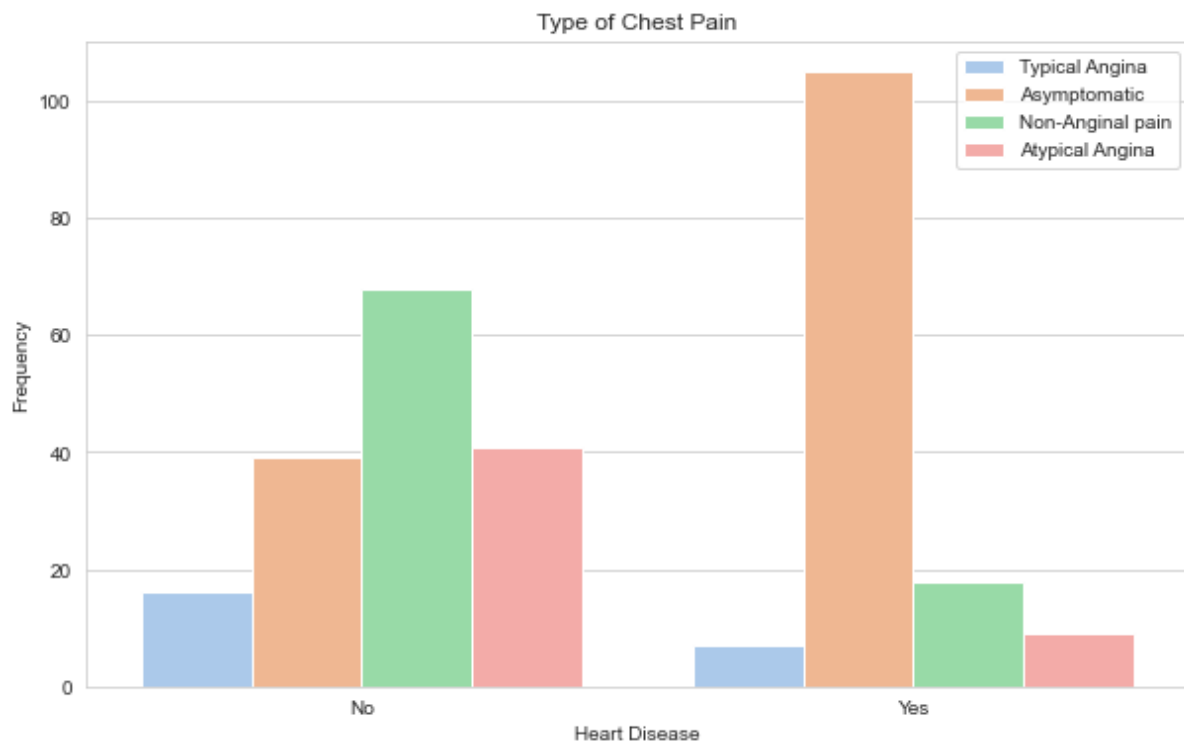
```
fig, ax = plt.subplots()
fig.set_size_inches(10, 6)
sns.histplot(x="Age", data=df, hue="AHD", palette="bright")
sns.set_style("whitegrid")
plt.title("Effect of age on frequency of heart disease")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.legend(["Heart Disease", "No Heart Disease"])
plt.show()
```



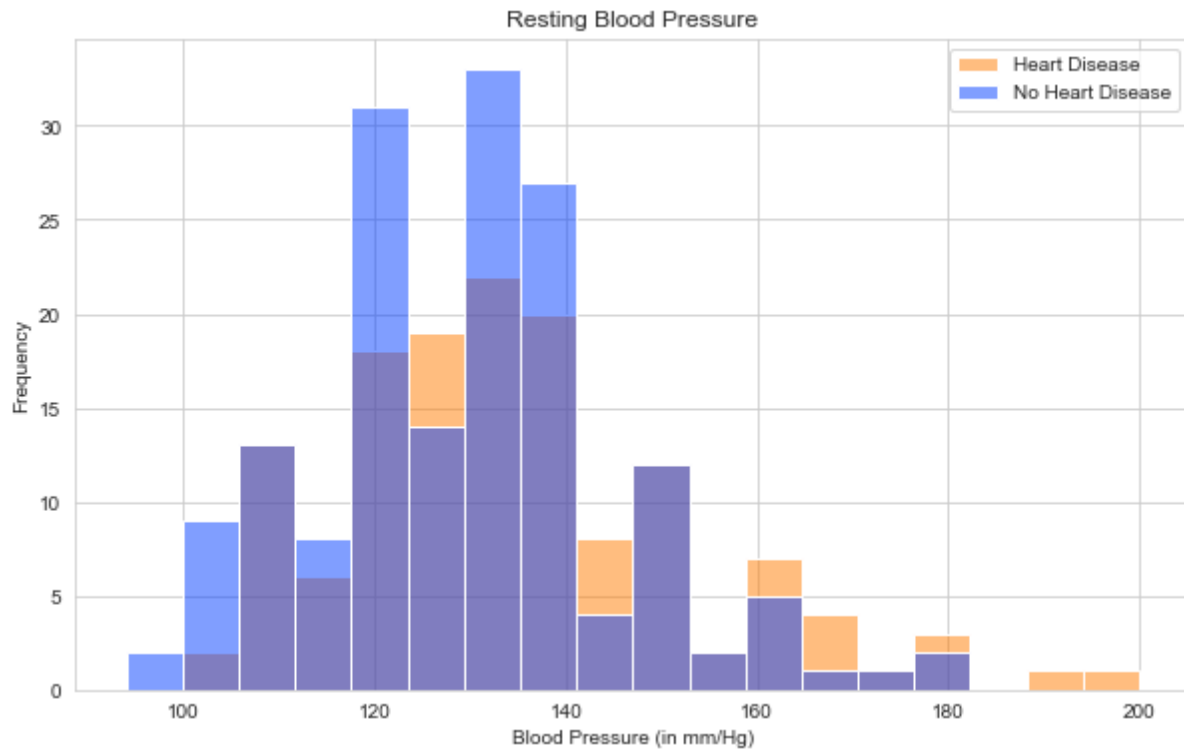
```
df['ChestPain'].unique()
```

```
array(['typical', 'asymptomatic', 'nonanginal', 'nontypical'],  
      dtype=object)
```

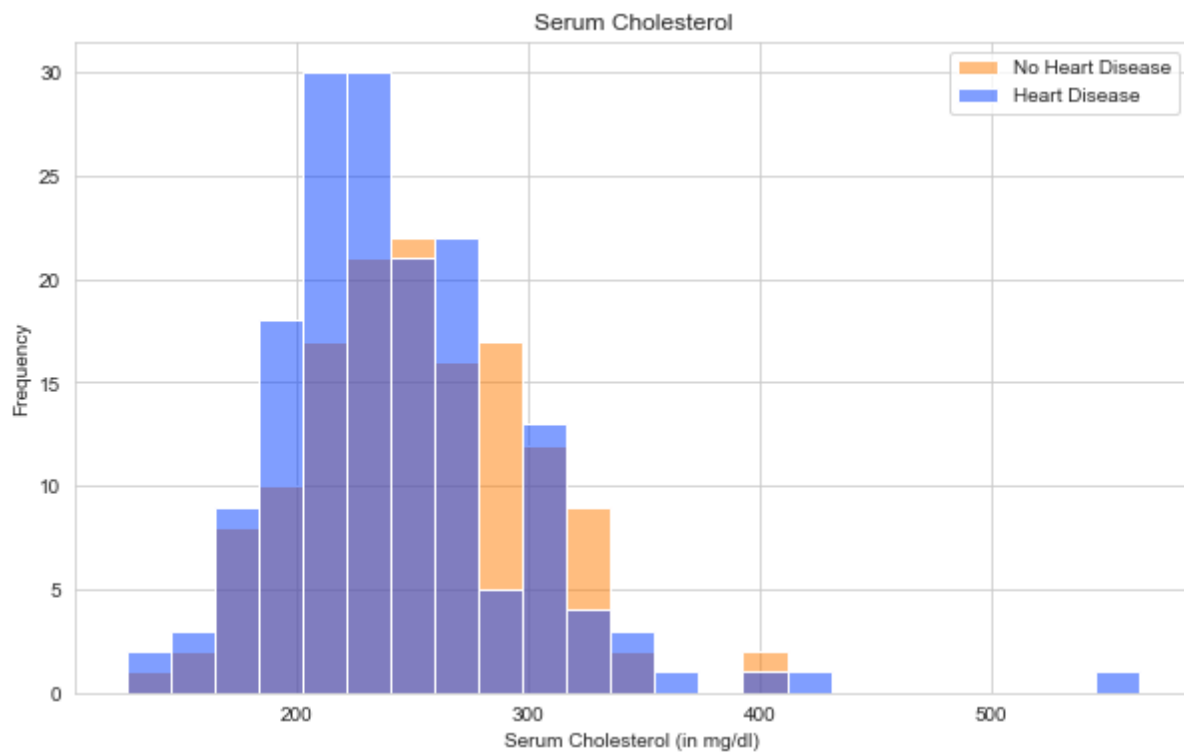
```
fig, ax = plt.subplots()
fig.set_size_inches(10, 6)
sns.countplot(x="AHD", hue="ChestPain", data=df, palette="pastel")
plt.title("Type of Chest Pain")
plt.xlabel("Heart Disease")
plt.ylabel("Frequency")
plt.legend(["Typical Angina", "Asymptomatic", "Non-Anginal pain", "Atypical Angina"])
plt.show()
```



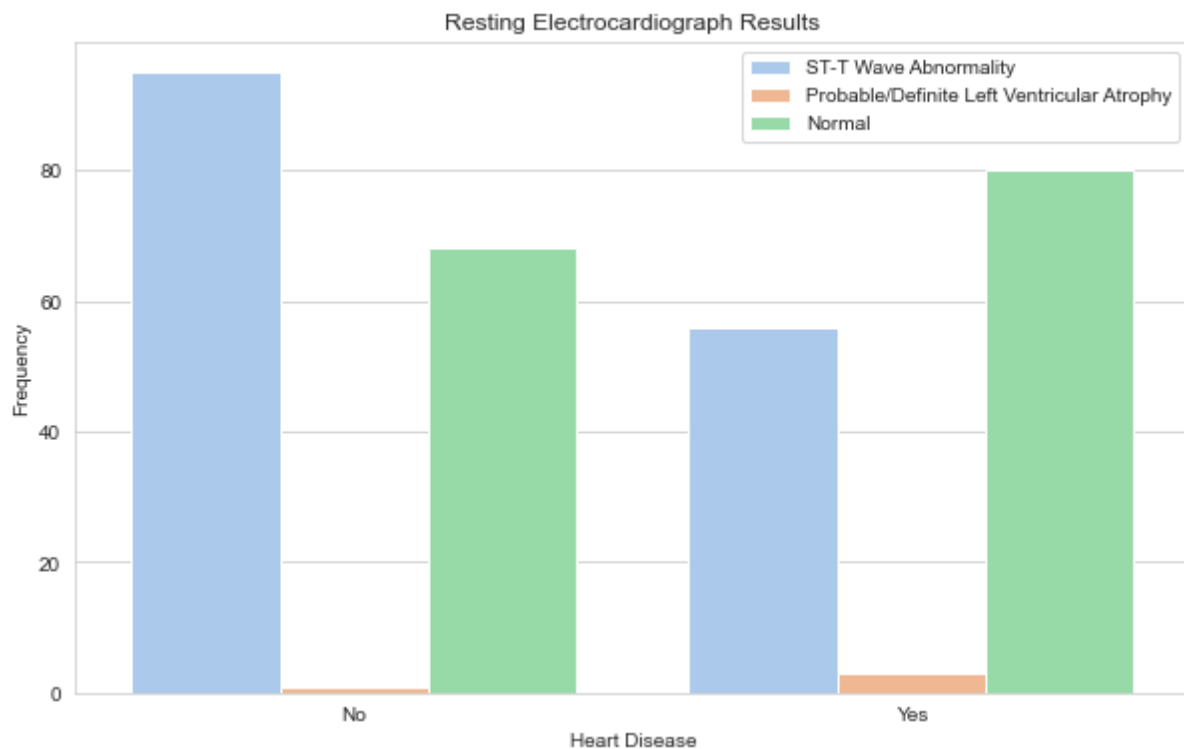
```
fig, ax = plt.subplots()
fig.set_size_inches(10, 6)
sns.histplot(x="RestBP" , data=df, hue="AHD", palette="bright")
sns.set_style("whitegrid")
plt.title("Resting Blood Pressure")
plt.xlabel("Blood Pressure (in mm/Hg)")
plt.ylabel("Frequency")
plt.legend(["Heart Disease", "No Heart Disease"])
plt.show()
```

```
fig, ax = plt.subplots()
fig.set_size_inches(10, 6)
sns.histplot(x="Chol" , data=df, hue="AHD", palette="bright")
sns.set_style("whitegrid")
plt.title("Serum Cholesterol")
plt.xlabel("Serum Cholesterol (in mg/dl)")
plt.ylabel("Frequency")
plt.legend(["No Heart Disease", "Heart Disease"])
plt.show()
```



```
fig, ax = plt.subplots()
fig.set_size_inches(10, 6)
sns.countplot(x="AHD", hue="RestECG", data=df, palette="pastel")
plt.title("Resting Electrocardiograph Results")
plt.xlabel("Heart Disease")
plt.ylabel("Frequency")
plt.legend(["ST-T Wave Abnormality", "Probable/Definite Left Ventricular Atrophy", "Normal"])
plt.show()
```



```
X = df[['Age','Sex','ChestPain','RestBP','Chol','RestECG','MaxHR']]
```

```
Y= df['AHD']
```

```
#X= df.values
```

```
#Y= df.AHD.values
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, Y_train, X_test, Y_test = train_test_split(X,Y,test_size=0.25)
```

```
X_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Int64Index: 227 entries, 245 to 231
```

```
Data columns (total 7 columns):
```

```
# Column Non-Null Count Dtype
```

```
--- ----
```

```
0 Age 227 non-null int64
```

```
1 Sex 227 non-null int64
```

```
2 ChestPain 227 non-null object
```

```
3 RestBP 227 non-null int64
```

```
4 Chol 227 non-null int64
```

```
5 RestECG 227 non-null int64
```

```
6 MaxHR    227 non-null  int64
dtypes: int64(6), object(1)
memory usage: 13.3+ KB
```

```
Y_train.info()
<class 'pandas.core.frame.DataFrame'>
Int64Index: 76 entries, 121 to 53
Data columns (total 7 columns):
#   Column    Non-Null Count  Dtype
---  -
0   Age       76 non-null    int64
1   Sex       76 non-null    int64
2   ChestPain 76 non-null    object
3   RestBP    76 non-null    int64
4   Chol      76 non-null    int64
5   RestECG   76 non-null    int64
6   MaxHR     76 non-null    int64
dtypes: int64(6), object(1)
memory usage: 4.5+ KB
```

Conclusion:

Data preparation is recognized for helping businesses and analytics to get ready and prepare the data for operations.

References:

[1]<https://medium.com/@learnbay/6-most-important-steps-for-data-preparation-in-machine-learning-61ae88ab8628>

Assignment No -5**Title: Regression technique****Problem Statement:**

This data consists of temperatures of INDIA averaging the temperatures of all place's month wise. Temperatures values are recorded in CELSIUS

- a) Apply Linear Regression and Logistic Regression using suitable library function and predict the Month-wise temperature.
- b) Assess the performance of regression models using MSE, MAE and R-Square metrics
- c) Visualize simple regression model.

Objective:

- To develop proficiency in preparing, analyzing, and preprocessing diverse real-world datasets, and applying appropriate machine learning techniques for predictive and descriptive analytics.
- To enhance the ability to evaluate machine learning models using relevant performance metrics and visualization techniques, enabling effective interpretation and decision-making in practical applications.

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows, Jupyter notebook.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15''Color Monitor, Keyboard, Mouse

References:

1. Ethem Alpaydin, Introduction to Machine Learning, PHI 2nd Edition, 2013.
2. Peter Flach: Machine Learning: The Art and Science of Algorithms that Make Sense of Data, Cambridge University Press, Edition 2012.

Theory:**Definition of Linear Regression**

In layman terms, we can define linear regression as **it is used for learning the linear relationship between the target and one or more forecasters**, and it is probably one of the most popular and well inferential algorithms in statistics. Linear regression endeavours to demonstrate the connection between two variables by fitting a linear equation to observed information. One variable is viewed as an explanatory variable, and the other is viewed as a dependent variable.

Types of Linear Regression

Normally, linear regression is divided into two types: Multiple linear regression and Simple linear regression.

1. Multiple Linear Regression

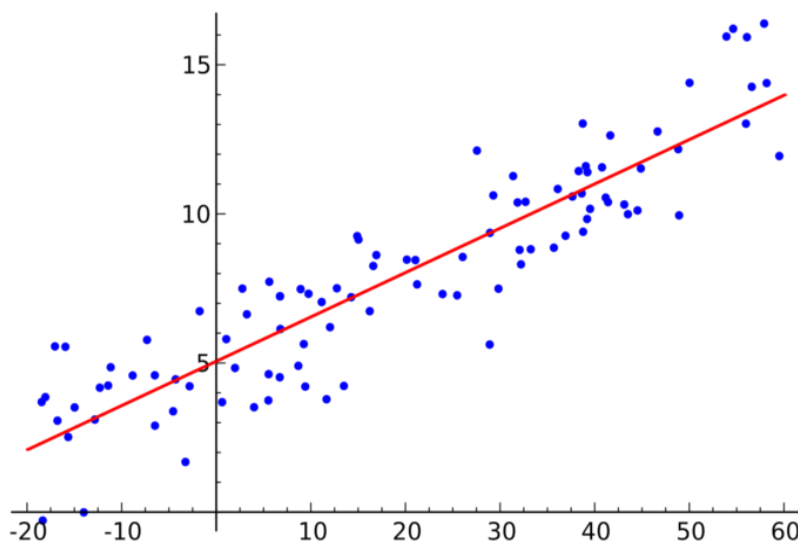
In this type of linear regression, we always attempt to discover the relationship between two or more independent variables or inputs and the corresponding dependent variable or output and the independent variables can be either continuous or categorical.

This linear regression analysis is very helpful in several ways like it helps in foreseeing trends, future values, and moreover predict the impacts of changes.

2. Simple Linear Regression

In simple linear regression, we aim to reveal the relationship between a single independent variable or you can say input, and a corresponding dependent variable or output. We can discuss this in a simple line as $y = \beta_0 + \beta_1 x + \epsilon$

Here, Y speaks to the output or dependent variable, β_0 and β_1 are two obscure constants that speak to the intercept and coefficient that is slope separately, and the error term is ϵ Epsilon. We can also discuss this in the form of a graph and here is a sample simple linear regression model graph.



Simple Linear Regression graph

What Actually is Simple Linear Regression?

It can be described as a method of statistical analysis that can be used to study the relationship between two quantitative variables.

Primarily, there are two things which can be found out by using the method of simple linear regression:

1. **Strength of the relationship between the given duo of variables.** (For example, the relationship between global warming and the melting of glaciers)
2. **How much the value of the dependent variable is at a given value of the independent variable.** (For example, the amount of melting of a glacier at a certain level of global warming or temperature)

Regression models are used for the elaborated explanation of the relationship between two given variables. There are certain types of regression models like logistic regression models, nonlinear regression models, and linear regression models. The linear regression model fits a straight line into the summarized data to establish the relationship between two variables.

Assumptions of Linear Regression

To conduct a simple linear regression, one has to make certain assumptions about the data. This is because it is a parametric test. The assumptions used while performing a simple linear regression are as follows:

- **Homogeneity of variance (homoscedasticity)**- One of the main predictions in a simple linear regression method is that the size of the error stays constant. This simply means that in the value of the independent variable, the error size never changes significantly.
- **Independence of observations**- All the relationships between the observations are transparent, which means that nothing is hidden, and only valid sampling methods are used during the collection of data.
- **Normality**- There is a normal rate of flow in the data.

These three are the assumptions of regression methods.

However, there is one additional assumption that has to be taken into consideration while specifically conducting a linear regression.

- **The line is always a straight line**- There is no curve or grouping factor during the conduction of a linear regression. There is a linear relationship between the variables (dependent variable and independent variable). If the data fails the assumptions of homoscedasticity or normality, a nonparametric test might be used. (For example, the Spearman rank test)

Example of data that fails to meet the assumptions: One may think that cured meat consumption and the incidence of colorectal cancer in the U.S have a linear relationship. But later on, it comes to the knowledge that there is a very high range difference between the collection of data of both the variables. Since the homoscedasticity assumption is being violated here, there can be no linear regression test. However, a Spearman rank test can be performed to know about the relationship between the given variables.

Applications of Simple Linear Regression

1. **Marks scored by students based on number of hours studied (ideally)**- Here marks scored in exams are dependent and the number of hours studied is independent.
2. **Predicting crop yields based on the amount of rainfall**- Yield is a dependent variable while the measure of precipitation is an independent variable.
3. **Predicting the Salary of a person based on years of experience**- Therefore, Experience becomes the independent while Salary turns into the dependent variable.

Limitations of Simple Linear Regression

Indeed, even the best information doesn't recount a total story. Regression investigation is ordinarily utilized in examination to set up that a relationship exists between variables. However, correlation isn't equivalent to causation: a connection between two variables doesn't mean one causes the other to occur. Indeed, even a line in a simple linear regression that fits the information focuses well may not ensure a circumstances and logical results relationship.

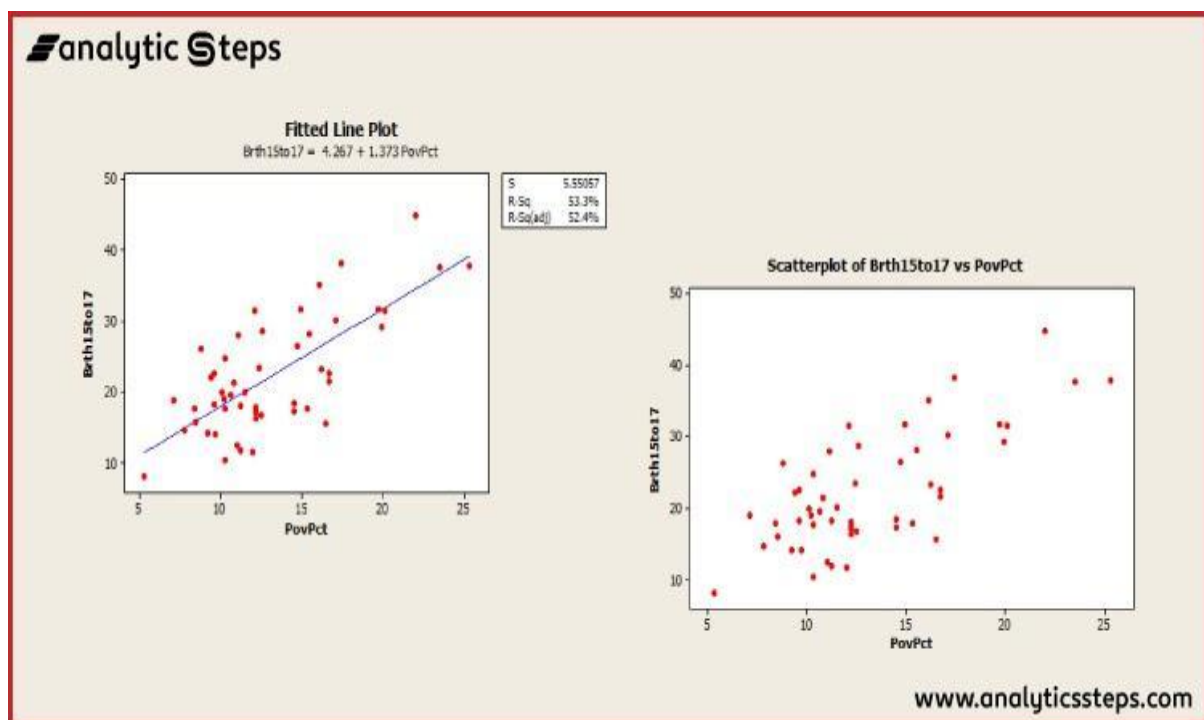
Utilizing a linear regression model will permit you to find whether a connection between variables exists by any means. To see precisely what that relationship is and whether one variable cause another, you will require extra examination and statistical analysis.

Examples of Simple Linear Regression

Now, let's move towards understanding simple linear regression with the help of an example. We will take an example of teen birth rate and poverty level data.

This dataset of size $n = 51$ is for the 50 states and the District of Columbia in the United States. The variables are y = year 2002 birth rate for each 1000 females 15 to 17 years of age and x = destitution rate, which is the percent of the state's populace living in families with wages underneath the governmentally characterized neediness level. (Information source: Mind On Statistics, 3rd version, Utts and Heckard).

Below is the graph (right image) in which you can see the (birth rate on the vertical) is indicating a normally linear relationship, on average, with a positive slope. As the poverty level builds, the birth rate for 15 to 17-year-old females will in general increment too.



Example graph of simple linear regression

Here is another graph (left graph) which is showing a regression line superimposed on the data.

The condition of the fitted regression line is given close to the highest point of the plot. The condition should express that it is for the "average" birth rate (or "anticipated" birth rate would be alright as well) as a regression condition portrays the normal estimation of y as a component of at least one x -variables. In statistical documentation, the condition could be composed $\hat{y} = 4.267 + 1.373x$.

- The interpretation of the slope (value = 1.373) is that the 15 to 17-year-old birth rate increases 1.373 units, on average, for each one unit (one per cent) increase in the poverty rate.
- The translation of the intercept (value=4.267) is that if there were states with a population rate = 0, the anticipated normal for the 15 to 17-year-old birth rate would be 4.267 for those states. Since there are no states with a poverty rate = 0 this understanding of the catch isn't basically significant for this model.
- In the chart with a regression line present, we additionally observe the data that $s = 5.55057$ and $r^2 = 53.3\%$.
- The estimation of s discloses to us generally the standard deviation of the contrasts between the y-estimations of individual perceptions and expectations of y dependent on the regression line. The estimation of r^2 can be deciphered to imply that destitution rates "clarify" 53.3% of the noticed variety in the 15 to 17-year-old normal birth paces of the states.

The R^2 (adj) value (52.4%) is a change in accordance with R^2 dependent on the number of x-variables in the model (just one here) and the example size. With just a single x-variable, the charged R^2 isn't significant.

Implementation:

#Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Importing the csv file

```
from google.colab import files
uploaded = files.upload()
```

#Read Student Grades .csv file and divide the data into dependent and independent variables.

```
data = pd.read_csv('Student_Grades_Data.csv')
data.head()
```

```
data.shape
```

```
(50, 2)
```

```
X = data.iloc[:, :-1].values
y = data.iloc[:, 1].values
```

```
X
```

```
array([[ 1],
       [ 5],
       [ 7],
       [ 3],
       [ 2],
       [ 9],
       [ 6],
       [12],
```

```
[11],
[ 2],
[ 4],
[ 8],
[13],
[ 9],
[14],
[10],
[ 6],
[12],
[ 1],
[ 4],
[14],
[10],
[11],
[ 4],
[ 5],
[ 8],
[ 1],
[ 2],
[ 3],
[ 7],
[ 8],
[14],
[ 7],
[ 8],
[ 1],
[ 2],
[ 3],
[ 4],
[ 5],
[ 6],
[ 7],
[ 8],
[ 9],
[10],
[11],
[12],
[13],
[14],
[ 8],
[ 2]])
```

Y

```
array([1.5, 2.7, 3.1, 2.1, 1.8, 3.9, 2.9, 4.5, 4.3, 1.8, 2.4, 3.5, 4.8,
       3.9, 5. , 4.1, 2.9, 4.5, 1.5, 2.4, 5. , 4.1, 4.3, 2.4, 2.7, 3.5,
       1.5, 1.8, 2.1, 3.1, 3.5, 5. , 3.1, 3.5, 1.5, 1.8, 2.1, 2.4, 2.7,
       2.9,  3.1,  3.5,  3.9,  4.1,  4.3,  4.5,  4.8,  5. ,  3.5,  1.8])
```

#Split the data into training and test datasets

from sklearn.model_selection **import** train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state = 0)

y_test

```
array([2.1, 3.5, 2.4, 3.5, 3.1, 1.8, 2.7, 5. , 4.3, 1.8, 3.5, 1.8, 1.5,
       1.5, 1.5])
```

#Fit the Simple Linear Regression Model

```
from sklearn.linear_model import LinearRegression
```

```
LinReg = LinearRegression()
```

```
LinReg.fit(X_train, y_train)
```

#Print the

```
print(f'a0 = {LinReg.intercept_}')
```

```
print(f'a1 = {LinReg.coef_}')
```

#Predicted grade scores from test dataset

```
y_predict = LinReg.predict(X_test)
```

```
y_predict
```

#Actual grade scores from test dataset

```
y_test
```

#Grades Vs Time to Study visualization on Training Data

```
plt.scatter(X_train, y_train, color='Blue')
```

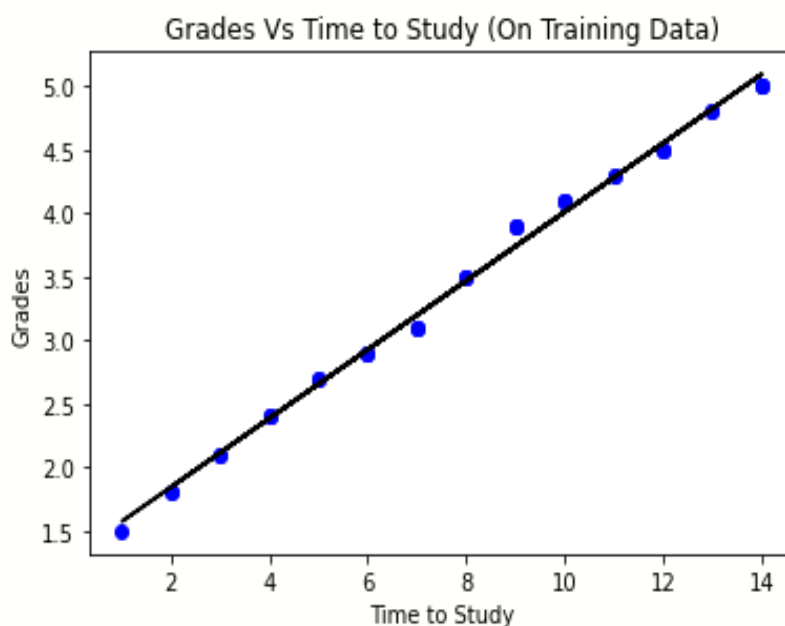
```
plt.plot(X_train, LinReg.predict(X_train), color='Black')
```

```
plt.title('Grades Vs Time to Study (On Training Data)')
```

```
plt.xlabel('Time to Study')
```

```
plt.ylabel('Grades')
```

```
plt.show()
```



#Grades Vs Time to Study visualization on Test Data

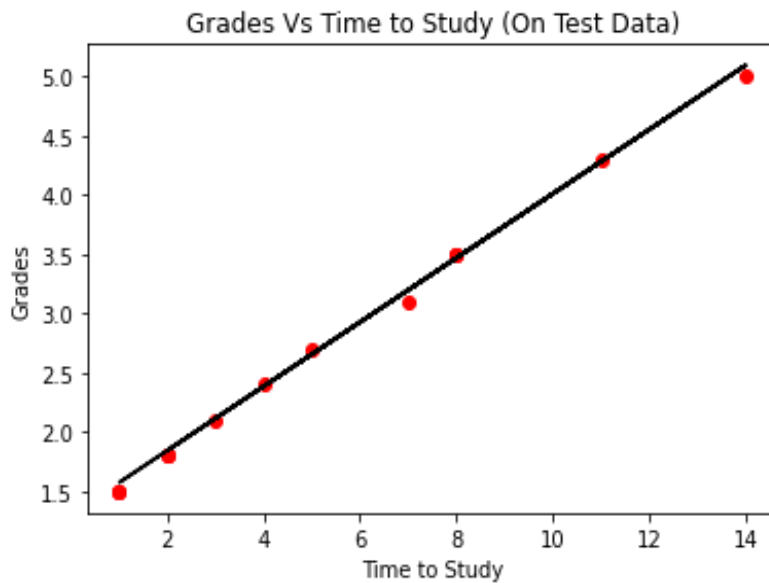
```
plt.scatter(X_test, y_test, color='Red')
```

```
plt.plot(X_train, LinReg.predict(X_train), color='Black')
```

```
plt.title('Grades Vs Time to Study (On Test Data)')
```

```
plt.xlabel('Time to Study')
```

```
plt.ylabel('Grades')
plt.show()
```



#Predicting Grade of a student when he studied for 10 Hrs. Example of how to pass an external value,
 #Independent of Test or Training Dataset

```
Predict_Grade = LinReg.predict([[10]])
Predict_Grade
```

#Model Evaluation using R-Square

```
from sklearn import metrics
r_square = metrics.r2_score(y_test, y_predict)
print('R-Square Error:', r_square)
```

#Model Evaluation using Mean Square Error (MSE)

```
print('Mean Squared Error:', metrics.mean_squared_error(y_test, y_predict))
```

#Model Evaluation using Root Mean Square Error (RMSE)

```
print('Root Mean Squared Error:', np.sqrt(metrics.mean_squared_error(y_test, y_predict)))
```

#Model Evaluation using Mean Absolute Error (MAE)

```
print('Mean Absolute Error:', metrics.mean_absolute_error(y_test, y_predict))
```

Conclusion

Simple linear regression is a regression model that figures out the relationship between one independent variable and one dependent variable using a straight line.

References:

[1]RiyaKumari,<https://www.analyticssteps.com/blogs/simple-linear-regression-applications-limitations-examples>

Assignment No -6

Title: Classification using Machine Learning

Problem Statement:

Perform following operations on given dataset:

- a) Apply Data pre-processing (Label Encoding, Data Transformation....) techniques if necessary.
- b) Perform data-preparation (Train-Test Split)
- c) Apply Decision tree classification Algorithm
- d) Evaluate Model.

Objective:

- To develop proficiency in preparing, analyzing, and preprocessing diverse real-world datasets, and applying appropriate machine learning techniques for predictive and descriptive analytics.
- To enhance the ability to evaluate machine learning models using relevant performance metrics and visualization techniques, enabling effective interpretation and decision-making in practical applications

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows , Jupyter notebook.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15''Color Monitor, Keyboard, Mouse

References:

1. *Ethem Alpaydin, Introduction to Machine Learning, PHI 2nd Edition, 2013.*
2. *Peter Flach: Machine Learning: The Art and Science of Algorithms that Make Sense of Data, Cambridge University Press, Edition 2012.*

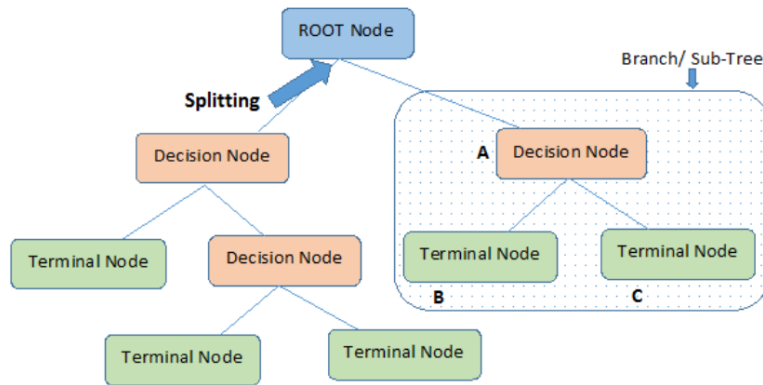
Theory:

Classification:

Classification is **a process of categorizing a given set of data into classes**, It can be performed on both structured or unstructured data. The process starts with predicting the class of data points given. The classes are often referred to as target, label or categories.

What is a Decision Tree?

It uses a flowchart like a tree structure to show the predictions that result from a series of feature-based splits. It starts with a root node and ends with a decision made by leaves.[1]



Root Nodes – It is the node present at the beginning of a decision tree. from this node the population starts dividing according to various features.

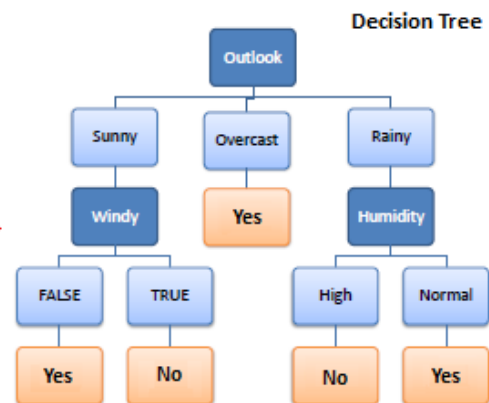
Decision Nodes – the nodes we get after splitting the root nodes are called Decision Node

Leaf Nodes – the nodes where further splitting is not possible are called leaf nodes or terminal nodes

Sub-tree – just like a small portion of a graph is called sub-graph similarly a sub-section of this decision tree is called sub-tree.

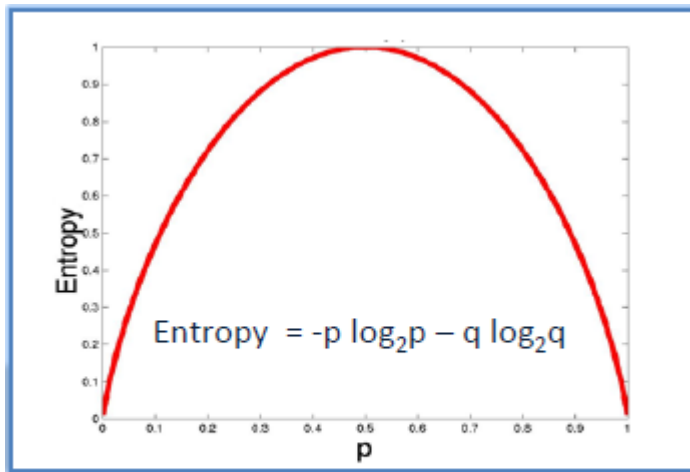
Pruning – It is cutting down some nodes to stop overfitting.

Predictors				Target
Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



Entropy:

Entropy is used to calculate the homogeneity of a sample. If the sample is completely homogeneous the entropy is zero and if the sample is equally divided it has entropy of one.



$$\text{Entropy} = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$$

a) Entropy using the frequency table of one attribute:

$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

Play Golf	
Yes	No
9	5

$$\begin{aligned} \text{Entropy}(\text{PlayGolf}) &= \text{Entropy}(5,9) \\ &= \text{Entropy}(0.36, 0.64) \\ &= -(0.36 \log_2 0.36) - (0.64 \log_2 0.64) \\ &= 0.94 \end{aligned}$$

b) Entropy using the frequency table of two attributes:

$$E(T, X) = \sum_{c \in X} P(c) E(c)$$

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14

$$\begin{aligned} E(\text{PlayGolf}, \text{Outlook}) &= P(\text{Sunny}) * E(3,2) + P(\text{Overcast}) * E(4,0) + P(\text{Rainy}) * E(2,3) \\ &= (5/14) * 0.971 + (4/14) * 0.0 + (5/14) * 0.971 \\ &= 0.693 \end{aligned}$$

Information Gain

The information gain is based on the decrease in entropy after a dataset is split on an attribute.

Constructing

a decision tree is all about finding attributes that return the highest information gain (i.e., the most homogeneous branches).[2]

Step 1: Calculate entropy of the target.

$$\begin{aligned}
 \text{Entropy(PlayGolf)} &= \text{Entropy}(5,9) \\
 &= \text{Entropy}(0.36, 0.64) \\
 &= -(0.36 \log_2 0.36) - (0.64 \log_2 0.64) \\
 &= 0.94
 \end{aligned}$$

Step 2: The dataset is then split on the different attributes. The entropy for each branch is calculated.

Then it is added proportionally, to get total entropy for the split. The resulting entropy is subtracted

from the entropy before the split. The result is the Information Gain, or decrease in entropy.

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
		Gain = 0.247	

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1
		Gain = 0.029	

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1
		Gain = 0.152	

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3
		Gain = 0.048	

$$Gain(T, X) = Entropy(T) - Entropy(T, X)$$

$$\begin{aligned}
 G(\text{PlayGolf}, \text{Outlook}) &= E(\text{PlayGolf}) - E(\text{PlayGolf}, \text{Outlook}) \\
 &= 0.940 - 0.693 = 0.247
 \end{aligned}$$

Step 3: Choose attribute with the largest information gain as the decision node, divide the dataset by its

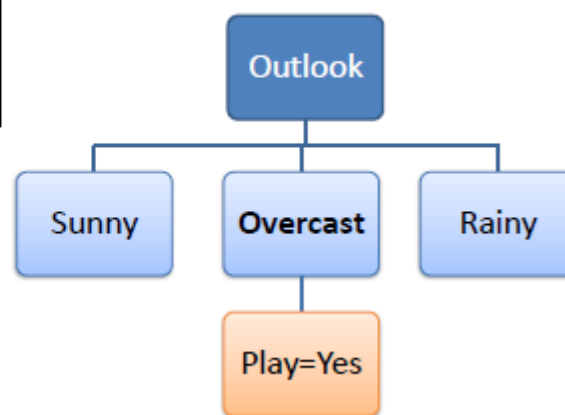
branches and repeat the same process on every branch.

		Play Golf	
		Yes	No
★ Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
		Gain = 0.247	

Outlook	Sunny	Outlook	Temp	Humidity	Windy	Play Golf
		Sunny	Mild	High	FALSE	Yes
		Sunny	Cool	Normal	FALSE	Yes
		Sunny	Cool	Normal	TRUE	No
		Sunny	Mild	Normal	FALSE	Yes
		Sunny	Mild	High	TRUE	No
	Overcast	Overcast	Hot	High	FALSE	Yes
Overcast		Cool	Normal	TRUE	Yes	
Overcast		Mild	High	TRUE	Yes	
Overcast		Hot	Normal	FALSE	Yes	
Rainy	Rainy	Hot	High	FALSE	No	
	Rainy	Hot	High	TRUE	No	
	Rainy	Mild	High	FALSE	No	
	Rainy	Cool	Normal	FALSE	Yes	
	Rainy	Mild	Normal	TRUE	Yes	

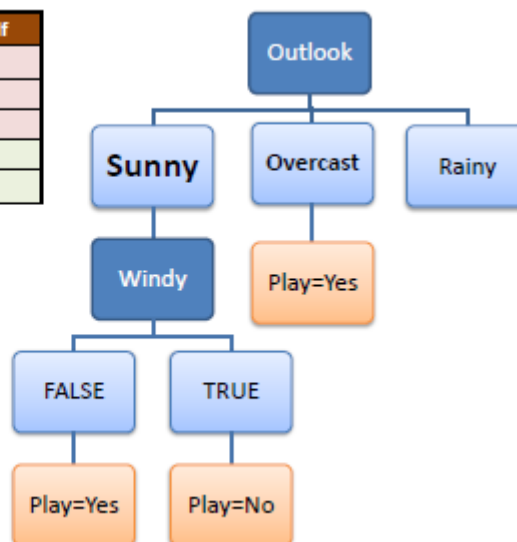
Step 4a: A branch with entropy of 0 is a leaf node.

Temp	Humidity	Windy	Play Golf
Hot	High	FALSE	Yes
Cool	Normal	TRUE	Yes
Mild	High	TRUE	Yes
Hot	Normal	FALSE	Yes



Step 4b: A branch with entropy more than 0 needs further splitting.

Temp	Humidity	Windy	Play Golf
Mild	High	FALSE	Yes
Cool	Normal	FALSE	Yes
Mild	Normal	FALSE	Yes
Cool	Normal	TRUE	No
Mild	High	TRUE	No



Step 5: The ID3 algorithm is run recursively on the non-leaf branches, until all data is classified.

Decision Tree to Decision Rules

A decision tree can easily be transformed to a set of rules by mapping from the root node to the leaf nodes one by one.

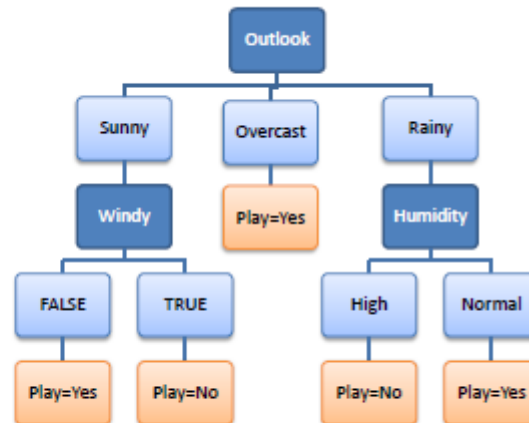
R₁: IF (Outlook=Sunny) AND (Windy=FALSE) THEN Play=Yes

R₂: IF (Outlook=Sunny) AND (Windy=TRUE) THEN Play=No

R₃: IF (Outlook=Overcast) THEN Play=Yes

R₄: IF (Outlook=Rainy) AND (Humidity=High) THEN Play=No

R₅: IF (Outlook=Rain) AND (Humidity=Normal) THEN Play=Yes



Pruning:

It is another method that can help us avoid overfitting. It helps in improving the performance of the tree by cutting the nodes or sub-nodes which are not significant. It removes the branches which have very low importance.

There are mainly 2 ways for pruning:

- Pre-pruning** – we can stop growing the tree earlier, which means we can prune/remove/cut a node if it has low importance **while growing** the tree.
- Post-pruning** – once our **tree is built to its depth**, we can start pruning the nodes based on their significance.

Implementation:

Importing all the necessary libraries

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib.pyplot as plt
```

Importing the csv file

```
df = pd.read_csv('../input/Admission_Predict.csv')
```

```
# Check null values in the dataset
```

```
df.isnull().sum()
```

```
Out[ ]:
Serial No.      0
GRE Score      0
TOEFL Score    0
University Rating 0
SOP            0
LOR            0
CGPA           0
Research       0
Chance of Admit 0
```

dtype: int64

df.columns

```
Index(['Serial No.', 'GRE Score', 'TOEFL Score', 'University Rating', 'SOP',
      'LOR ', 'CGPA', 'Research', 'Chance of Admit '],
      dtype='object')
```

Updating chance of admission column
 # if chance >= 80% CHANCE = 1
 # if chance < 80% CHANCE = 0

```
dataset.loc[dataset['Chance of Admit '] < 0.8, 'Chance of Admit '] = 0
dataset.loc[dataset['Chance of Admit '] >= 0.8, 'Chance of Admit '] = 1
```

Initializing the variables

```
X = df.drop(['Chance of Admit ', 'Serial No.'], axis=1)
y = df['Chance of Admit ']
```

Split the data into training and testing set

```
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size=0.25, random_state=123)
```

```
# importing required libraries
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics
```

```
# Creating Decision Tree classifier object
clf = DecisionTreeClassifier()
```

```
# Training Decision Tree Classifier
clf = clf.fit(X_train, Y_train)
```

```
#Predicting for the test data
y_pred = clf.predict(X_test)
```

Confusion matrix:

```
print("confusion matrix:\n")
print(metrics.confusion_matrix(Y_test, y_pred))
```

confusion matrix:

```
[[79  3]
 [ 4 39]]
```

```
print("1. Accuracy Score:", metrics.accuracy_score(Y_test, y_pred))
```

```
print("2. Precision Score:", metrics.precision_score(Y_test, y_pred))
```

```
print("3. Recall Score:", metrics.recall_score(Y_test, y_pred))
```

```
print("4. f1 Score:", metrics.f1_score(Y_test, y_pred))
```

1. Accuracy Score: 0.944
2. Precision Score: 0.9285714285714286
3. Recall Score: 0.9069767441860465
4. f1 Score: 0.9176470588235294

Support Vector Machine (SVM):

Support Vector Machine (SVM) is a supervised machine learning algorithm used for both **classification** and **regression** problems. It is primarily used for classification tasks because of its high accuracy and ability to work well with high-dimensional data. SVM aims to find the optimal boundary, called a **hyperplane**, that separates data points belonging to different classes with the maximum possible margin.

Working Principle of SVM

The main objective of SVM is to determine the best decision boundary (hyperplane) that separates the classes.

- For a **2-dimensional dataset**, the decision boundary is a **line**.
- For a **3-dimensional dataset**, it is a **plane**.
- For datasets with more than three features, it is called a **hyperplane**.

Among all possible hyperplanes, SVM selects the one that **maximizes the margin**, i.e., the distance between the hyperplane and the nearest data points from each class.

Key Concepts of SVM

1. Hyperplane

A **hyperplane** is a decision boundary that separates different classes in the feature space.

For example:

- Students likely to be admitted lie on one side of the hyperplane.
- Students unlikely to be admitted lie on the other side.

Mathematically, a hyperplane is represented as:

$$w^T x + b = 0$$

where:

- w = weight vector
- x = feature vector
- b = bias (intercept)

2. Support Vectors

Support vectors are the data points that lie closest to the hyperplane. These points are the most influential in determining the position and orientation of the decision boundary. If support vectors change, the hyperplane also changes.

3. Margin

The **margin** is the distance between the hyperplane and the nearest data points of both classes. SVM attempts to maximize this margin because a larger margin generally improves the model's ability to classify unseen data.

4. Kernel Function

Many real-world datasets are not linearly separable. In such cases, SVM uses **kernel functions** to transform the data into a higher-dimensional space where a linear separation becomes possible.

Common kernel functions include:

Kernel	Description
Linear Kernel	Used when data is linearly separable.
Polynomial Kernel	Captures polynomial relationships between features.
Radial Basis Function (RBF) Kernel	Handles complex non-linear relationships; most commonly used.
Sigmoid Kernel	Similar to neural network activation functions.

For the Graduate Admission dataset, the **RBF kernel** is generally preferred because the relationship between student attributes and admission decisions is often non-linear.

Advantages of SVM

- High classification accuracy.
- Effective for high-dimensional datasets.
- Works well with both linear and non-linear data.
- Less prone to overfitting due to margin maximization.
- Performs well even when the number of features is large.

Limitations of SVM

- Computationally expensive for very large datasets.
- Sensitive to the choice of kernel and hyperparameters.
- Feature scaling is necessary for good performance.
- Less interpretable compared to simpler models like Decision Trees.

Applications of SVM

Support Vector Machine is widely used in various domains, including:

- Student admission prediction
- Email spam detection
- Medical diagnosis
- Face recognition
- Handwritten digit recognition
- Text and sentiment classification
- Credit risk assessment
- Bioinformatics and gene classification

SVM Workflow

1. Collect the dataset.
2. Perform data preprocessing (handle missing values, encode categorical data if required, and standardize features).
3. Split the dataset into training and testing sets.
4. Train the SVM classifier using the training data.
5. Predict the class labels for the test data.
6. Evaluate the model using metrics such as Accuracy, Confusion Matrix, Precision, Recall, and F1-score.

Conclusion

Support Vector Machine (SVM) is a powerful supervised learning algorithm that constructs an optimal decision boundary to classify data into different categories. Its ability to maximize the margin and use kernel functions for non-linear classification makes it highly effective for predicting graduate admissions based on students' academic performance. With proper preprocessing and parameter tuning, SVM achieves high accuracy and generalizes well to unseen data, making it an excellent choice for admission prediction tasks.

Build SVM Model

```
svm=SVC(kernel='rbf')
```

```
svm.fit(X_train,y_train)
```

Prediction

```
y_pred=svm.predict(X_test)
```

Model Evaluation**Accuracy**

```
accuracy=accuracy_score(y_test,y_pred)
```

```
print("Accuracy:",accuracy)
```

Confusion Matrix

```
cm=confusion_matrix(y_test,y_pred)
```

```
print(cm)
```

Visualization

```
sns.heatmap(cm,  
             annot=True,  
             cmap='Blues',  
             fmt='d')
```

```
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.title("Confusion Matrix")  
plt.show()
```

Classification Report

```
print(classification_report(y_test,y_pred))
```

Application:

Helpful in solving classification problems.

References:

[1]Anshul Saini ,Analytics Vidhya, Decision Tree Algorithm – A Complete Guide

[2]Dr. Saed Sayad, https://www.saedsayad.com/decision_tree.htm

[3] Christopher M. Bishop, Pattern Recognition and Machine Learning, Springer (Special Indian Edition), 2007,ISBN: 978-9380501413.

Assignment No -7

Title: Implement any two clustering algorithm

Problem Statement:

Perform following operations on given dataset:

- a) Apply Data pre-processing (Label Encoding, Data Transformation....) techniques if necessary.
- b) Perform data-preparation (Train-Test Split)
- c) Apply Machine Learning Algorithm
- d) Evaluate Model.
- e) Apply Cross-Validation and Evaluate Model

Objective:

- To develop proficiency in preparing, analyzing, and preprocessing diverse real-world datasets, and applying appropriate machine learning techniques for predictive and descriptive analytics.
- To enhance the ability to evaluate machine learning models using relevant performance metrics and visualization techniques, enabling effective interpretation and decision-making in practical applications.

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows , Jupyter notebook.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15''Color Monitor, Keyboard, Mouse

References:

3. *Ethem Alpaydin, Introduction to Machine Learning, PHI 2nd Edition, 2013.*
4. *Peter Flach: Machine Learning: The Art and Science of Algorithms that Make Sense of Data, Cambridge University Press, Edition 2012..*

Theory:

Clustering Algorithm

Clustering is an **unsupervised machine learning** technique used to group similar data points into clusters based on their characteristics or features. Unlike supervised learning, clustering does not require labeled data. The main objective is to maximize the similarity among data points within the same cluster while minimizing the similarity between different clusters.

Clustering is widely used in data mining, pattern recognition, image processing, customer segmentation, document classification, medical diagnosis, and anomaly detection.

Characteristics of Clustering

- It is an unsupervised learning technique.
- No predefined class labels are required.
- Groups similar data points into clusters.
- Maximizes intra-cluster similarity.
- Minimizes inter-cluster similarity.
- Useful for exploratory data analysis.

Types of Clustering Algorithms**1. Partitioning Clustering**

- K-Means
- K-Medoids (PAM)

2. Hierarchical Clustering

- Agglomerative Clustering
- Divisive Clustering

3. Density-Based Clustering

- DBSCAN
- OPTICS

4. Model-Based Clustering

- Gaussian Mixture Models (GMM)

5. Grid-Based Clustering

- STING
- CLIQUE

Applications of Clustering

- Customer segmentation
- Market basket analysis
- Image segmentation
- Medical diagnosis
- Social network analysis
- Document clustering
- Fraud detection
- Recommendation systems
- Bioinformatics

Introduction to K-means Clustering

K-means clustering is a type of unsupervised learning, which is used when you have unlabeled data (i.e., data without defined categories or groups). The goal of this algorithm is to find groups in the data, with the number of groups represented by the variable K.

The algorithm works iteratively to assign each data point to one of K groups based on the features that are provided. Data points are clustered based on feature similarity.

The results of the K-means clustering algorithm are:

1. The centroids of the K clusters, which can be used to label new data
2. Labels for the training data (each data point is assigned to a single cluster) Rather than defining groups before looking at the data, clustering allows you to find and analyze the groups that have formed organically.

The "Choosing K" section below describes how the number of groups can be determined. Each centroid of a cluster is a collection of feature values which define the resulting groups.

Examining the centroid feature weights can be used to qualitatively interpret what kind of group each cluster represents.

This introduction to the K-means clustering algorithm covers:

- Common business cases where K-means is used
- The steps involved in running the algorithm

Some examples of use cases are:

- Behavioral segmentation:
 - o Segment by purchase history
 - o Segment by activities on application, website, or platform.
 - o Define personas based on interests
 - o Create profiles based on activity monitoring
- Inventory categorization:
 - o Group inventory by sales activity
 - o Group inventory by manufacturing metrics
- Sorting sensor measurements:
 - o Detect activity types in motion sensors
 - o Group images
 - o Separate audio
 - o Identify groups in health monitoring
- Detecting bots or anomalies:
 - o Separate valid activity groups from bots
 - o Group valid activity to clean up outlier detection

In addition, monitoring if a tracked data point switches between groups over time can be used to detect meaningful changes in the data.

Algorithm:

The K-means clustering algorithm uses iterative refinement to produce a final result. The algorithm inputs are the number of clusters K and the data set. The data set is a collection of features for each data point. The algorithms start with initial estimates for the K centroids, which can either be randomly generated or randomly selected from the data set. The algorithm then iterates between two steps:

1. Data assignment step:

Each centroid defines one of the clusters. In this step, each data point is assigned to its nearest centroid, based on the squared Euclidean distance. More formally, if c_i is the collection of centroids in set C, then each data point x is assigned to a cluster based on

$$\operatorname{argmin}_{c_i \in C} \operatorname{dist}(c_i, x)^2$$

where $\operatorname{dist}(\cdot)$ is the standard (L2) Euclidean distance. Let the set of data point assignments for each i th cluster centroid be S_i .

2. Centroid update step:

In this step, the centroids are recomputed. This is done by taking the mean of all data points assigned to that centroid's cluster.

$$c_i = \frac{1}{|S_i|} \sum_{x_i \in S_i} x_i$$

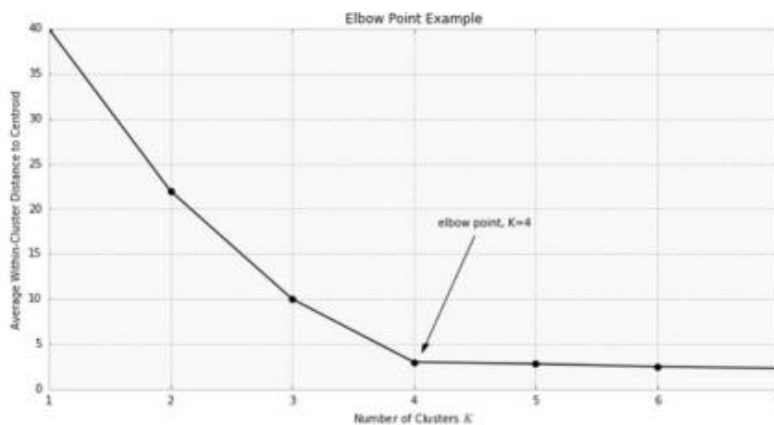
The algorithm iterates between steps one and two until a stopping criteria is met (i.e., no data points change clusters, the sum of the distances is minimized, or some maximum number of iterations is reached).

This algorithm is guaranteed to converge to a result. The result may be a local optimum (i.e. not necessarily the best possible outcome), meaning that assessing more than one run of the algorithm with randomized starting centroids may give a better outcome.

Choosing K

The algorithm described above finds the clusters and data set labels for a particular pre-chosen K. To find the number of clusters in the data, the user needs to run the K-means clustering algorithm for a range of K values and compare the results. In general, there is no method for determining exact value of K, but an accurate estimate can be obtained using the following techniques. One of the metrics that is commonly used to compare results across different values of K is the mean distance between data points and their cluster centroid. Since increasing the number of clusters will always reduce the distance to data points, increasing K will always decrease this metric, to the extreme of reaching zero when K is the same as the number of data points. Thus, this metric cannot be used as the sole target. Instead, mean distance to the centroid as a function of K is plotted and the "elbow point," where the rate of decrease sharply shifts, can be used to roughly determine K.

A number of other techniques exist for validating K, including cross-validation, information criteria, the information theoretic jump method, the silhouette method, and the G-means algorithm. In addition, monitoring the distribution of data points across groups provides insight into how the algorithm is splitting the data for each K.



Implementation:

Importing Dataset

```
from pandas import read_csv
A=read_csv("E:/DS1/Mall_Customers.csv")
```

Dropping the irrelevant columns

```
B=A.drop(["Customer_ID"],axis=1)
```

Label encoding

```
# Import label encoder
from sklearn import preprocessing

# label_encoder object knows how to understand word labels.
label_encoder = preprocessing.LabelEncoder()

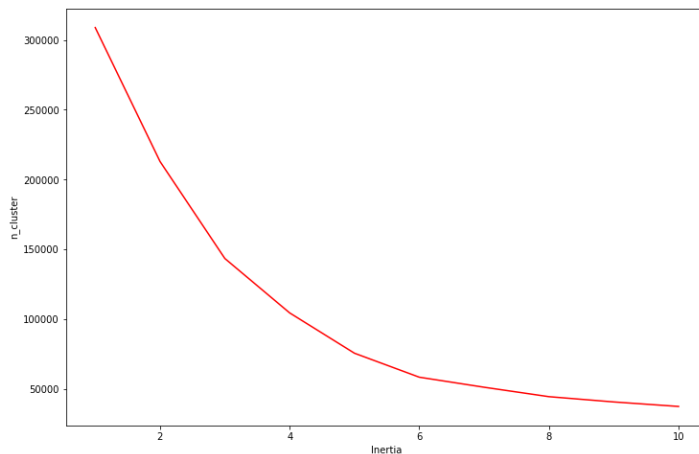
# Encode labels in column 'species'.
B['Genre']= label_encoder.fit_transform(B['Genre'])

B['Genre'].unique()
array([1, 0], dtype=int64)
```

Finding K

```
from sklearn.cluster import KMeans
cluster = []
for k in range(1, 11):
    kmean = KMeans(n_clusters=k).fit(B)
    cluster.append(kmean.inertia_)
```

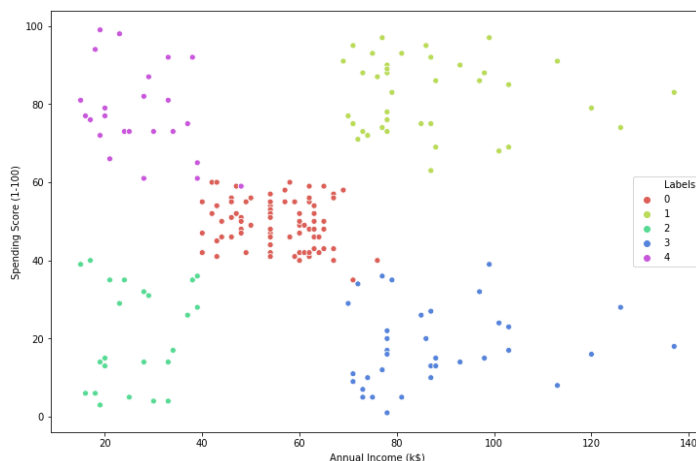
```
import matplotlib.pyplot as plt
plt.figure(figsize=(12, 8))
plt.plot(range(1, 11), cluster, 'r-')
plt.xlabel('Inertia')
plt.ylabel('n_cluster')
plt.show()
```



With above value of K, Create K means clustering model

```
km = KMeans(n_clusters=5).fit(B)
B['Labels'] = km.labels_
```

```
import seaborn as sns
plt.figure(figsize=(12, 8))
sns.scatterplot(B['Annual Income (k$)'], B['Spending Score (1-100)'], hue=B['Labels'],
               palette=sns.color_palette('hls', 5))
plt.show()
```



Hierarchical Clustering

Hierarchical Clustering is an unsupervised learning algorithm that creates a hierarchy of clusters. Instead of partitioning the dataset into a fixed number of clusters, it builds a tree-like structure called a dendrogram, which represents the relationships among data points.

Hierarchical clustering is useful when the number of clusters is unknown and when understanding the nested grouping of data is important.

Types of Hierarchical Clustering

1. Agglomerative Hierarchical Clustering (Bottom-Up)

- Initially, each data point forms its own cluster.
- The two closest clusters are merged repeatedly.
- The process continues until all points belong to a single cluster or the desired number of clusters is obtained.

This is the most commonly used hierarchical clustering method.

2. Divisive Hierarchical Clustering (Top-Down)

- Starts with all data points in one cluster.
- Recursively splits the clusters into smaller clusters.
- Continues until each data point becomes an individual cluster or the required number of clusters is achieved.

Working of Agglomerative Hierarchical Clustering

1. Treat each data point as an individual cluster.
2. Compute the distance between all clusters.
3. Merge the two closest clusters.
4. Update the distance matrix.
5. Repeat until only one cluster remains or the desired number of clusters is formed.
6. Use a dendrogram to determine the optimal number of clusters.

Linkage Methods

The distance between clusters is determined using different linkage criteria.

1. Single Linkage

- Distance between the nearest data points of two clusters.
- May produce elongated clusters (chaining effect).

2. Complete Linkage

- Distance between the farthest data points of two clusters.
- Produces compact clusters.

3. Average Linkage

- Average distance between all pairs of data points from two clusters.

4. Ward's Linkage

- Merges clusters that result in the smallest increase in within-cluster variance.
- Often produces balanced and compact clusters.

Dendrogram

A dendrogram is a tree-like diagram that illustrates the sequence in which clusters are merged or split during hierarchical clustering. By drawing a horizontal cut across the dendrogram, the desired number of clusters can be identified.

Advantages of Hierarchical Clustering

- No need to specify the number of clusters initially.
- Produces a dendrogram for easy visualization.
- Can identify nested clusters.
- Works well for small to medium-sized datasets.

- Suitable for both numerical and categorical data (with appropriate distance measures).

Limitations of Hierarchical Clustering

- Computationally expensive for large datasets.
- Sensitive to noise and outliers.
- Once clusters are merged or split, the decision cannot be undone.
- Performance depends on the choice of distance metric and linkage method.

Applications of Hierarchical Clustering

- Customer segmentation
- Gene expression analysis
- Medical diagnosis
- Document clustering
- Image analysis
- Social network analysis
- Recommendation systems
- Biological taxonomy

Difference Between K-Means and Hierarchical Clustering

Feature	K-Means Clustering	Hierarchical Clustering
Learning Type	Unsupervised	Unsupervised
Number of Clusters	Must be specified in advance	Can be determined using a dendrogram
Output	Flat clusters	Hierarchical tree (dendrogram)
Speed	Faster	Slower
Suitable for	Large datasets	Small to medium datasets
Initialization	Requires initial centroids	No centroid initialization required
Reassignment	Data points can change clusters	Merging/splitting decisions are irreversible

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
df = pd.read_csv("Mall_Customers.csv")
df.head()
df.info()
print(df.isnull().sum())
```

Select Features

Use Annual Income and Spending Score.

```
X = df.iloc[:, [3,4]].values
```

or

```
X = df[['Annual Income (k$)',
        'Spending Score (1-100)']]
```

Step B: Apply Hierarchical Clustering

Draw Dendrogram

```
plt.figure(figsize=(10,6))
```

```
dendrogram(linkage(X,
                    method='ward'))
```

```
plt.title("Dendrogram")
plt.xlabel("Customers")
plt.ylabel("Euclidean Distance")
```

```
plt.show()
```

Choose the number of clusters from the dendrogram (typically 5).

Train Hierarchical Clustering Model

```
hc = AgglomerativeClustering(
    n_clusters=5,
    metric='euclidean',
    linkage='ward'
)
```

```
y_pred = hc.fit_predict(X)
```

Step C: Evaluate Model

Silhouette Score

```
score = silhouette_score(X, y_pred)
```

```
print("Silhouette Score:", score)
```

A silhouette score closer to 1 indicates better clustering quality.

Step D: Visualize Clusters

```
plt.figure(figsize=(8,6))
```

```
plt.scatter(X[y_pred==0,0],
            X[y_pred==0,1],
            s=80,
            label='Cluster 1')
```

```
plt.scatter(X[y_pred==1,0],
            X[y_pred==1,1],
            s=80,
            label='Cluster 2')
```

```
plt.scatter(X[y_pred==2,0],
            X[y_pred==2,1],
            s=80,
            label='Cluster 3')
```

```
plt.scatter(X[y_pred==3,0],
            X[y_pred==3,1],
            s=80,
            label='Cluster 4')
```

```
plt.scatter(X[y_pred==4,0],
```

```
X[y_pred==4,1],  
s=80,  
label='Cluster 5')  
  
plt.title("Customer Segmentation using Hierarchical Clustering")  
  
plt.xlabel("Annual Income (k$)")  
plt.ylabel("Spending Score (1-100)")  
  
plt.legend()  
  
plt.show()
```

References:

- [1] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- [2] Andrea Trevino, Introduction to K-means Clustering , Oracle AI & Data Science Blog

Assignment No -8**Title: Association Rule Learning****Problem Statement:**

Download Market Basket Optimization dataset from below link.

Data Set: <https://www.kaggle.com/hemanthkumar05/market-basket-optimization>

This dataset comprises the list of transactions of a retail company over the period of one week. It contains a total of 7501 transaction records where each record consists of the list of items sold in one transaction. Using this record of transactions and items in each transaction, find the association rules between items.

There is no header in the dataset and the first row contains the first transaction, so mentioned header = None here while loading dataset.

- a. Follow following steps :
- b. Data Preprocessing
- c. Generate the list of transactions from the dataset
- d. Train Apriori algorithm on the dataset
- e. Visualize the list of rules

Generated rules depend on the values of hyper parameters. By increasing the minimum confidence value and find the rules accordingly

Objective:

- To develop proficiency in preparing, analyzing, and preprocessing diverse real-world datasets, and applying appropriate machine learning techniques for predictive and descriptive analytics.
- To enhance the ability to evaluate machine learning models using relevant performance metrics and visualization techniques, enabling effective interpretation and decision-making in practical applications.

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows , Jupyter notebook.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15''Color Monitor, Keyboard, Mouse

References:

- 1.Ethem Alpaydin, Introduction to Machine Learning, PHI 2nd Edition,2013.
2. Peter Flach: Machine Learning: The Art and Science of Algorithms that Make Sense of Data,Cambridge University Press, Edition 2012.

Theory:

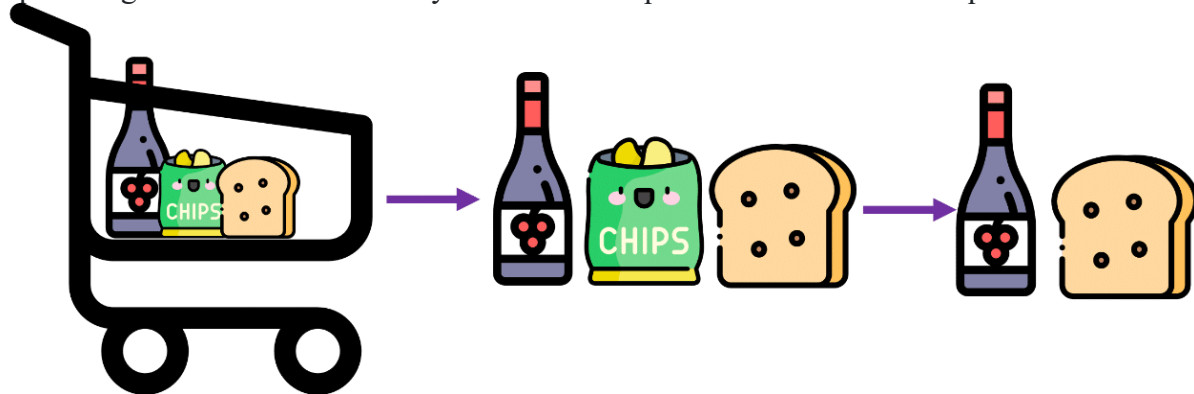
the Apriori algorithm is used for the purpose of [association rule mining](#). Association rule mining is a technique to identify frequent patterns and associations among a set of items.

For example, understanding customer buying habits. By finding correlations and associations between different items that customers place in their 'shopping basket,' recurring patterns can be derived.

This process of identifying an association between products/items is called association rule mining. To implement association rule mining, many algorithms have been developed. Apriori algorithm is one of the most popular and arguably the most efficient algorithms among them.

Apriori Algorithm

Apriori algorithm assumes that any subset of a frequent itemset must be frequent.



Say, a transaction containing {wine, chips, bread} also contains {wine, bread}. So, according to the principle of Apriori, if {wine, chips, bread} is frequent, then {wine, bread} must also be frequent.

The key concept in the Apriori algorithm is that it assumes all subsets of a frequent itemset to be frequent. Similarly, for any infrequent itemset, all its supersets must also be infrequent.

Here is a dataset consisting of six transactions in an hour. Each transaction is a combination of 0s and 1s, where 0 represents the absence of an item and 1 represents the presence of it.

Here is a dataset consisting of six transactions in an hour. Each transaction is a combination of 0s and 1s, where 0 represents the absence of an item and 1 represents the presence of it.

Transaction ID	Wine	Chips	Bread	Milk
1	1	1	1	1
2	1	0	1	1
3	0	0	1	1
4	0	1	0	0
5	1	1	1	1

6	1	1	0	1
---	---	---	---	---

We can find multiple rules from this scenario. For example, in a transaction of wine, chips, and bread, if wine and chips are bought, then customers also buy bread.

$\{\text{wine, chips}\} \Rightarrow \{\text{bread}\}$

In order to select the interesting rules out of multiple possible rules from this small business scenario, we will be using the following measures:

- Support
- Confidence
- Lift
- Conviction

Support

Support of item x is nothing but the ratio of the number of transactions in which item x appears to the total number of transactions.

$$\text{i.e., Support(wine)} = \frac{\text{Number of transactions in which the item wine appears}}{\text{Total number of transactions}}$$

$$\text{Support(wine)} = 4/6 = 0.66667$$

Confidence

Confidence ($x \Rightarrow y$) signifies the likelihood of the item y being purchased when item x is purchased. This method takes into account the popularity of item x .

$$\text{i.e., Conf}(\{\text{wine, chips}\} \Rightarrow \{\text{bread}\}) = \frac{\text{support(wine,chips,bread)}}{\text{support(wine,chips)}}$$

$$\text{Conf}(\{\text{wine, chips}\} \Rightarrow \{\text{bread}\}) = \frac{2}{6 \div \frac{3+4}{6}} = 0.667$$

Lift

Lift ($x \Rightarrow y$) is nothing but the ‘interestingness’ or the likelihood of the item y being purchased when item x is sold. Unlike confidence ($x \Rightarrow y$), this method takes into account the popularity of the item y .

i.e.,

$$\text{lift}(\{\text{wine, chips}\} \Rightarrow \{\text{bread}\}) = \frac{\text{support}(\text{wine, chips, bread})}{\text{support}(\text{wine, chips})}$$

$$\text{lift}(\{\text{wine, chips}\} \Rightarrow \{\text{bread}\}) = \frac{\frac{2}{6}}{\frac{3+4}{6}} = 1$$

- Lift ($x \Rightarrow y$) = 1 means that there is no correlation within the itemset.
- Lift ($x \Rightarrow y$) > 1 means that there is a positive correlation within the itemset, i.e., products in the itemset, x and y , are more likely to be bought together.
- Lift ($x \Rightarrow y$) < 1 means that there is a negative correlation within the itemset, i.e., products in itemset, x and y , are unlikely to be bought together.

Dataset

Below is the transaction data from Day 1. This dataset contains 6 items and 22 transaction records.

	A	B	C	D	E	F
1	Wine	Chips	Bread	Butter	Milk	Apple
2	Wine		Bread	Butter	Milk	
3			Bread	Butter	Milk	
4		Chips				Apple
5	Wine	Chips	Bread	Butter	Milk	Apple
6	Wine	Chips			Milk	
7	Wine	Chips	Bread	Butter		Apple
8	Wine	Chips			Milk	
9	Wine		Bread			Apple
10	Wine		Bread	Butter	Milk	
11		Chips	Bread	Butter		Apple
12	Wine			Butter	Milk	Apple
13	Wine	Chips	Bread	Butter	Milk	
14	Wine		Bread		Milk	Apple
15	Wine		Bread	Butter	Milk	Apple
16	Wine	Chips	Bread	Butter	Milk	Apple
17		Chips	Bread	Butter	Milk	Apple
18		Chips		Butter	Milk	Apple
19	Wine	Chips	Bread	Butter	Milk	Apple
20	Wine		Bread	Butter	Milk	Apple
21	Wine	Chips	Bread		Milk	Apple
22		Chips				

Environment Setup:

Before we move forward, we need to install the 'apyori' package first.

```
pip install apyori
```

Market Basket Analysis Implementation within Python

With the help of the apyori package, we will be implementing the Apriori algorithm in order to help the manager in market basket analysis.

Step 1: Import the libraries

```
In [1]: #Importing the required datasets
import numpy as np
import pandas as pd
from apyori import apriori
```

Step 2: Load the dataset

```
In [2]: #Loading the dataset
store_data = pd.read_csv('Day1.csv', header=None)
```

Step 3: Have a glance at the records

```
In [3]: #Having a glance at the records
store_data
```

```
out[3]:
```

	0	1	2	3	4	5
0	Wine	Chips	Bread	Butter	Milk	Apple
1	Wine	NaN	Bread	Butter	Milk	NaN
2	NaN	NaN	Bread	Butter	Milk	NaN
3	NaN	Chips	NaN	NaN	NaN	Apple
4	Wine	Chips	Bread	Butter	Milk	Apple
5	Wine	Chips	NaN	NaN	Milk	NaN
6	Wine	Chips	Bread	Butter	NaN	Apple
7	Wine	Chips	NaN	NaN	Milk	NaN
8	Wine	NaN	Bread	NaN	NaN	Apple
9	Wine	NaN	Bread	Butter	Milk	NaN
10	NaN	Chips	Bread	Butter	NaN	Apple
11	Wine	NaN	NaN	Butter	Milk	Apple
12	Wine	Chips	Bread	Butter	Milk	NaN
13	Wine	NaN	Bread	NaN	Milk	Apple
14	Wine	NaN	Bread	Butter	Milk	Apple
15	Wine	Chips	Bread	Butter	Milk	Apple
16	NaN	Chips	Bread	Butter	Milk	Apple
17	NaN	Chips	NaN	Butter	Milk	Apple
18	Wine	Chips	Bread	Butter	Milk	Apple
19	Wine	NaN	Bread	Butter	Milk	Apple
20	Wine	Chips	Bread	NaN	Milk	Apple
21	NaN	Chips	NaN	NaN	NaN	NaN

Step 4 :Convert Pandas DataFrame into a list of lists

```
In [5]: #Converting the pandas dataframe into a list of lists
records = []
for i in range(0, 22):
    records.append([str(store_data.values[i,j]) for j in range(0, 6)])
```

Step 5: Build the Apriori model

```
In [7]: #Building the first apriori model
association_rules = apriori(records, min_support=0.50, min_confidence=0.7, min_lift=1.2, min_length=2)
association_results = list(association_rules)
```

Step 6: Print out the number of rules

```
In [8]: #Getting the number of rules
print(len(association_results))
```

Step 7: Have a glance at the rule

```
In [10]: #Glancing at the first rule
         print(association_results)
```

```
[RelationRecord(items=frozenset({'Milk', 'Butter', 'Bread'}), support=0.5, ordered_statistics=[OrderedStatistic(items_base=frozenset({'Milk', 'Bread'}), items_add=frozenset({'Butter'}), confidence=0.8461538461538461, lift=1.241025641025641)])]
```

The support value for the first rule is 0.5. This number is calculated by dividing the number of transactions containing 'Milk,' 'Bread,' and 'Butter' by the total number of transactions.

The confidence level for the rule is 0.846, which shows that out of all the transactions that contain both "Milk" and "Bread", 84.6 % contain 'Butter' too.

The lift of 1.241 tells us that 'Butter' is 1.241 times more likely to be bought by the customers who buy both 'Milk' and 'Butter' compared to the default likelihood sale of 'Butter.'

Conclusion:

Apriori algorithm is implemented